ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

ESP™ Programmer's Manual

DSM 00180-01

© 1995 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
00	June 1982
01	September 1985

ESP Programmer's Manual

To re-order this document, request part number DSO-00180-00

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This Document was written by Robert Shelton.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VER-A-TEL
VIDEOTRAX			

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
DART	ESP	MULTI	*in*Sight/am
*in*Front/am			

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

TABLE OF CONTENTS

CHAPTER ONE - INTRODUCTION.....1-1
OTHER DOCUMENTATION 1-1
GRAPHIC CONVENTIONS 1-2

CHAPTER TWO - PROGRAMMING GUIDELINES..... 2-1
PROGRAM FLOW 2-1
Initialization..... 2-1
Allocate Memory Pool.....2-1
Load Screen.....2-2
Open Screen for Processing.....2-2
Set Up Defaults.....2-2
Process User Input..... 2-2
Return Values.....2-2
Finish Up 2-3

CHAPTER THREE - ASSEMBLY LANGUAGE SUBROUTINES.....3-1
GENERAL GUIDELINES.....3-1
Prerequisites.....3-1
Calling Format 3-2
Preserving Registers.....3-2
Flag Names and Types.....3-2
TBX$ALERT - DISPLAY MESSAGE TEXT 3-3
TBX$ALWFLD - ALLOW EDITING OF ESP FIELD 3-3
TBX$CLRFLS - CLEAR A BLOCK OF FIELDS.....3-4
TBX$CLSSCR - CLOSE ESP SCREEN 3-4
TBX$DSPERR - DISPLAY MESSAGE TEXT 3-5
TBX$DSPSCR - UPDATE ESP SCREEN DISPLAY 3-5
TBX$FETSCR - LOAD ESP SCREEN FROM DISK 3-6
TBX$GETBNM - GET BINARY VALUE 3-7
TBX$GETCHR - GET NEXT CHARACTER 3-7
TBX$GETDAT - GET DATE VALUE 3-8
TBX$GETFV6 - GET FLOATING POINT VALUE 3-8
TBX$GETPUI - GET POP-UP ITEM NUMBER 3-9
TBX$GETSCR - PROCESS ESP SCREEN.....3-9
TBX$GETSTR - GET STRING VALUE 3-11
TBX$GETYNF - GET YES/NO VALUE 3-11

TBX$GTUSER - GET USER PARAMETERS 3-12
Options.....3-12
TBX$HIDFLD - HIDE ESP FIELD.....3-13
TBX$MEMALC - ALLOCATE MEMORY PACKET 3-14
TBX$MEMCHG - CHANGE SIZE OF MEMORY PACKET 3-14
TBX$MEMDEA - DEALLOCATE MEMORY PACKET 3-15
TBX$MEMFRE - GET MEMORY POOL FREE SIZE 3-15
TBX$MEMINI - INITIALIZE MEMORY POOL 3-16
TBX$MPLALC - ALLOCATE PACKET IN SHARED MEMORY POOL 3-16
TBX$MPLDEA - DEALLOCATE PACKET IN SHARED MEMORY POOL 3-17
TBX$MPLFRE - FIND FREE AREA OF SHARED MEMORY POOL 3-17
TBX$OPNSCR - OPEN ESP SCREEN 3-18
TBX$SCRPRT - PRINT ESP SCREEN 3-18
TBX$SETBNM - SET BINARY VALUE 3-19
TBX$SETFV6 - SET FLOATING POINT VALUE 3-20
TBX$SETPUI - SELECT POP-UP ITEM 3-20
TBX$SETSTR - SET STRING VALUE 3-21
TBX$SETYNF - SET YES/NO VALUE 3-21
TBX$SHWFLD - SHOW ESP FIELD.....3-22
TBX$SKPFLD - SKIP ESP FIELD 3-22
TBX$SQFLSZ - SEQUENTIAL FILE SIZE 3-23
TBX$SQFSZ1.....3-24
TBX$STUSER - SET USER PARAMETERS 3-24
Options.....3-24
TBX$TRMCAP - GET TERMINAL CAPABILITIES 3-25
TBX$TRMINI - INITIALIZE TERMINAL SESSION 3-26
Status Line Formats.....3-26

CHAPTER FOUR - AlphaBASIC XCALLS..... 4-1
ALERT - DISPLAY ERROR MESSAGE 4-1
BUSY - ACCESS BUSY INDICATOR 4-1
CLRBLK - CLEAR BLOCK ON TERMINAL SCREEN 4-2
CLRFLS - CLEAR FIELDS ON ESP SCREEN 4-3
CLSSCR - CLOSE ESP SCREEN 4-3
ERRDSP - DISPLAY ERROR MESSAGE 4-3
FETCH - LOAD SCREEN DEFINITION FILE 4-4
FILINB - READ ONE BYTE FROM A SEQUENTIAL FILE 4-5
GETFSC - GET FIELD SECURITY CLEARANCE STATUS 4-5
Read Security Test.....4-6
Write Security Test..... 4-6
GETPUI - GET POP-UP ITEM 4-6
GETUSR - GET USER IMPURE AREA ELEMENTS 4-6
GETUSR Elements..... 4-7
GETVAL - GET VALUES FROM SCREEN FIELDS 4-9
GETYNF - GET YES/NO RESPONSE FROM FIELD 4-10
GMENU - GENERALIZED MENU PROCESSING 4-11
Designing a Menu Screen to Use With GMENU.....4-11
GTCHR - GET ONE CHARACTER FROM KEYBOARD 4-13
GTSCR - HANDLE SCREEN PROCESSING..... 4-13
Special Processing Flags.....4-14
Returning to the Application.....4-15
Return Caused by Undefined Character.....4-16

Return Caused by Screen-generated Interrupt 4-16
Screen Output 4-16
Redisplaying a Screen.....4-17
HIDFLD - HIDE SCREEN FIELDS 4-17
INITRM - INITIALIZE THE TERMINAL 4-17
Status Lines.....4-18
INPREC - READ RECORD FROM SEQUENTIAL FILE 4-19
IREAD - READ AN ISAM DATA RECORD 4-19
ISMROK - READ THE ISAM ROCK VALUES 4-20
JULCVT - CONVERT DATE TO/FROM JULIAN.....4-21
JUSTXT - JUSTIFY A STRING 4-21
LSTBOX - PROCESS A LIST BOX 4-22
Designing a Screen to Use With LSTBOX.....4-23
Implementing LSTBOX in Your Program..... 4-23
Using a List Box Screen.....4-24
OPNSCR - OPEN A SCREEN FOR PROCESSING 4-25
OPNTMP - GET TEMPORARY FILE EXTENSION.....4-25
PRTSCR - PRINT AN IMAGE OF THE SCREEN 4-25
PRTSEL - SELECT A PRINTER..... 4-27
RREAD - READ RANDOM FILE RECORD 4-28
SELFLG - PROCESS SELECTION SCREEN 4-28
The SELFLG Screen File.....4-29
The Field Flag Table..... 4-29
SETDFC - SET SPECIAL DISPLAY CHARACTERISTICS 4-30
SETDFP - CHANGE A FIELD'S SCREEN POSITION 4-31
SETDSP - SET ALL FIELDS FOR REDISPLAY 4-31
SETFSC - SET FIELD SECURITY 4-32
SETPUI - SET POP-UP ITEM 4-32
SETUSR - SET USER IMPURE ELEMENTS 4-32
SETUSR Elements..... 4-33
SETVAL - SET VALUES INTO SCREEN FIELDS 4-36
SETYNF - SET A YES/NO RESPONSE INTO A FIELD 4-37
SKPFLD - SKIP SCREEN FIELDS 4-37
TOOLBX - MISCELLANEOUS TOOLBX FUNCTIONS 4-38
TBX'FLDCNT: Number of Screen Fields.....4-38
TBX'GETOLH: Get On-line Help Message 4-38
TBX'SQFLSZ: Sequential File Sizing.....4-38

CHAPTER FIVE - SPECIAL KEY HANDLING..... 5-1
WHEN YOU RETURN TO YOUR APPLICATION 5-1
CHARACTERS INTERPRETED BY ESP.....5-1
Function Key Translation..... 5-2
Hints for Using the ESP Translation Table..... 5-3
The CHRCHK Macro..... 5-4
Using Your Own Function Key Translations.....5-5

CHAPTER SIX - TRANSLATING ESP'S MESSAGES..... 6-1
CHANGING THE MESSAGES..... 6-1
TRANSLATING THE MESSAGE FILE..... 6-1
ASSEMBLING THE MESSAGE FILE.....6-2

APPENDIX A - TOOLBOX MESSAGE CODES..... A-1

APPENDIX B - THE USER IMPURE AREA.....B-1
BLOCK LINKS.....B-1
USER OPTIONS..... B-2
WORK AREAS..... B-2
ACCUMULATORS.....B-3
EXPRESSION STACK AREA..... B-3
PRINTER DEFINITIONS.....B-3
TRANSLATION TABLE..... B-5
CURRENT SCREEN INFORMATION.....B-5
BUSY INDICATOR VARIABLES..... B-5
SIGNIN INFORMATION..... B-6
INDIRECT INPUT INFORMATION.....B-6
EXTERNAL LINKAGES..... B-6
MISCELLANEOUS FLAGS.....B-7
KEYSTROKE RECORD INFORMATION.....B-7
GRAPHICS DEVICE DEFINITIONS..... B-7
REGISTER STORAGE..... B-7
EIGHT-BIT CHARACTER STORAGE.....B-8

DOCUMENT HISTORY

INDEX

READER’S COMMENTS

CHAPTER ONE
INTRODUCTION

This manual is for software designers who want to use ESP in developing their applications. It describes how to access the subroutines contained in the ESP toolbox file, TOOLBX.SYS. These subroutines let you interface screens or reports you design using SEDIT, the ESP screen design program, to your application programs.

This manual contains five chapters and one appendix:

- Chapter One introduces the manual and explains the graphic conventions it uses.
- Chapter Two gives programming guidelines and general information.
- Chapter Three is for assembly language programmers—it describes the toolbox subroutines you can call from your own programs.
- Chapter Four is for the AlphaBASIC programmer—it describes the AlphaBASIC XCALLs included with ESP so you can use the toolbox routines from your BASIC programs.
- Chapter Five lists the function key definitions ESP uses and describes when your application needs to handle keyboard input itself instead of having ESP do it.
- Appendix A lists the toolbox message codes.

OTHER DOCUMENTATION

This book describes how to interface ESP screens and reports to your application. Other aspects of ESP are covered in these manuals:

- *Overview of ESP* gives a general description of ESP's capabilities and operation.
- The *SEDIT User's Manual* shows how to use the SEDIT program to design screens and report formats.

GRAPHIC CONVENTIONS

In order to make this document as easy as possible to understand, we use several conventions in layout and type style, so you can recognize whether we are talking about a command or key, when you're to type an entry, and so on.

CONVENTION	EXPLANATION
FILE.EXT	AMOS command names and file names, such as TOOLBX.SYS, are shown in all capitals.
Screen Editor	ESP screen names are printed in the same type face as regular text, with initial capitals.
`What You Type`	Text you are expected to type is shown in a bold Courier typeface. Text you should type exactly as shown is in all capitals; variable responses or parts of commands, such as file names, are in lower case.
`PROGRAM SAMPLES`	Programming examples and statement formats are shown in Courier typeface, as is text AMOS or ESP displays on your screen.
LIST	ESP command names are shown in small, bold capitals.
KEY	Keys you need to press are shown in a keycap box.
KEY 1 / KEY 2	Two keys separated by a slash, like this, means you need to press them at the same time. Hold down the first key and press the second one.
	The halt symbol marks important information you should read carefully before continuing. This often indicates something you MUST or MUST NOT do, so read carefully.
	The hint symbol marks a helpful bit of information. It often points out a short cut which may save you time or trouble.
	The remember symbol marks information you need to keep in mind while you are following a set of instructions.

CHAPTER TWO
PROGRAMMING GUIDELINES

This chapter describes the general flow of a program which gets input from an ESP screen—when to use which toolbox features and other considerations for you to keep in mind when writing an application using ESP.

PROGRAM FLOW

Except where stated, all subroutine names are the names of AlphaBASIC XCALL subroutines. If you are programming in assembly language, use the equivalent assembly language subroutine names. All ESP assembly language subroutines start with TBX$.

INITIALIZATION

The first thing an application program using ESP should do is XCALL INITRM. This clears the screen, sets up top and bottom status lines, selects seven-bit or eight-bit character mode and sets many other terminal options. After INITRM, your program is ready to interface with ESP.

ALLOCATE MEMORY POOL

If you are programming in assembly language and need some memory, you can allocate a large memory pool all at one time, then use ESP's memory management routines to allocate and deallocate packets within it as required. Here is a typical example of allocating an initial memory pool:

```
PUSH    size
PUSH
GETMEM  @SP
POP     A6
POP     D6
TOOLBX  TBX$MEMINI, SJT, A0, PSH
```

You can then use the TBX$MEMALC and TBX$MEMDEA routines to allocate and deallocate modules from within that memory pool. You could use these two routines to manage screen memory, for example.

LOAD SCREEN

Next, you load the ESP screen file from disk using XCALL FETCH for BASIC or TBX$FETSCR for assembly language.

OPEN SCREEN FOR PROCESSING

Once the screen is loaded, use OPNSCR to tell ESP about it. This puts the screen in a list of open screens so ESP can properly handle the windowing of screens on top of one another.

SET UP DEFAULTS

Before displaying the screen, you can set field defaults using SETVAL or change field characteristics using SETDFC. There are also assembly language routines to let you hide, show or skip fields (TBX$SHWFLD, TBX$HIDFLD, and TBX$SKPFLD).

In most cases, you will not need to change field characteristics or set defaults within your program code. You can perform these functions when you define the screen using SEDIT. See the *SEDIT User's Guide*.

PROCESS USER INPUT

Once the screen is ready for processing, use XCALL GTSCR to process the user's keyboard input. GTSCR handles the actual displaying of the screen and processes all user input.

When you call GTSCR you pass it the field number to begin processing at and a starting editing character, usually [CTRL] / [U] so the cursor starts at the beginning of the field. ESP sees this character just before it starts accepting keyboard input. As another example, you can use GTSCR just to display a screen by passing [MENU] as the starting editing character. ESP processes [MENU] as if the user pressed it, paints the screen, and returns immediately to the calling program. This is how the DSPSCR XCALL works.

RETURN VALUES

As GTSCR processes, it updates the variable containing the screen. When GTSCR returns to your program, your processing routines can use GETVAL and the other value retrieval calls to retrieve the contents of the screen from the variable. You can then process the information in any way you want.

When GTSCR returns to your program, it returns the field it was on and the character causing the return. You need to check how and why it returned. Subsequent action depends on your screen and the design of your program. You should check for [MENU] and [EXECUTE], you may have to look for pre- or post-edit interrupts or function keys, and, finally, ignore anything your program doesn't recognize.

You should store the field and character returned by GTSCR. They control the field you go to if you reenter GTSCR to continue processing the same screen. For example, if you reset the editing character to [CTRL] / [U] before returning to GTSCR, editing resumes on the last field processed. If you exited a screen with a [RETURN] on a post-edit interrupt, and use the [RETURN] to reenter GTSCR, editing resumes at the next field. If it was [BACK TAB], you resume at the field before the one you left.

FINISH UP

When your program has finished processing, it should call INITRM again before it exits, this time with no options. Seeing this, INITRM clears the screen and the status lines and resets the terminal back into seven-bit mode and normal keyboard input.

Any memory you allocated and initialized with TBX$MEMINI is deallocated by AMOS because you initially allocated it using an AMOS monitor call.

CHAPTER THREE
ASSEMBLY LANGUAGE SUBROUTINES

This chapter describes the assembly language subroutines available to you in the ESP toolbox file, TOOLBX.SYS. You can call these subroutines in any of your assembly language programs.

This chapter assumes you understand how to program in Alpha Micro assembly language. For more information on programming in assembly language, see the *Assembly Language Programmer's Manual*.

Before using any of these subroutines, read the guidelines described in Chapter 2 of this manual.

The subroutines are listed in alphabetical order.

GENERAL GUIDELINES

The following sections include information which applies to all of the toolbox routines.

PREREQUISITES

Any program which uses any of the toolbox subroutines must include a `SEARCH TOOLBX.UNV` statement.

Before using any of the toolbox subroutines, you must check to make sure there is a valid version of the toolbox loaded into system memory. To do this, you use the CHKTBX call, in one of these formats

```
CHKTBX
```

or:

```
CHKTBX ABTMSG
```

If you include ABTMSG, any error displays an appropriate message and exits. If you don't include ABTMSG, you may detect the following conditions:

```
BEQ  TBX.OK    ; Toolbox is installed and compatible.
BCS  OLD.OS    ; Pre 1.3D AMOS, toolbox not supported.
BMI  NO.TBX    ; Toolbox is not installed.
BVS  OLD.TBX   ; Newer version of toolbox required.
```

CALLING FORMAT

All of the toolbox subroutines are invoked using the TOOLBX directive. The basic calling format is:

```
TOOLBX TBX$name
```

Some subroutines require an extended calling format. The extended formats are:

```
TOOLBX TBX$name,SJT
```

This format uses a subroutine jump table, using register A6.

```
TOOLBX TBX$name,SJT,Ax
```

This format uses a subroutine jump table, using register Ax, where ***x*** can be any register number you want. This format is required when A6 is being used for other information. Whenever this format is required, we suggest you use:

```
TOOLBX TBX$name,SJT,Ax,PSH
```

Including the PSH preserves the contents of register Ax, which are destroyed otherwise. This is the format we show for all routines which require the software jump table with a register specification.

The particular format for each subroutine is listed in the section on that subroutine.

PRESERVING REGISTERS

All toolbox routines follow the accepted AMOS rules for use and preservation of registers. Specifically, unless specified otherwise, registers A6, D6, and D7 may not be preserved during any of these routines, and you should not count on their contents being unchanged afterward.

FLAG NAMES AND TYPES

Many of these routines allow you to set various flags or return flag values. All flag symbols include either a dollar sign ($) or a percent sign (%).

- $ indicates the symbol defines an octal value.
- % indicates the symbol defines a bit number.

Be sure to use the procedures for the correct type of flag when setting or checking these values.

TBX$ALERT - DISPLAY MESSAGE TEXT

TBX$ALERT displays a message and waits for the user to press any key before clearing the message and continuing. The message text is taken from the system message table, the toolbox message table, or the current application message table, depending on the message number. If there is no message for the specified number, the message displayed is:

```
Message number xxx ... press any key to continue
```

AMOS system message numbers start at 1. See the *AMOS Monitor Calls Manual* for symbol names for AMOS error messages.

Toolbox message numbers start at 20000. This is defined as TETBS%.

User defined messages start at 22500. This is defined as UMTBS%.

TBX$ALERT sets the N-flag to 1 if the user presses MENU to acknowledge the message. Your application can then abort the current access if appropriate.

INPUTS

D6 = Message number.

FORMAT

```
TOOLBX TBX$ALERT,SJT
```

OUTPUTS

Z-flag = 1: User acknowledged by pressing a key.
N-flag = 1: User pressed MENU.

TBX$ALWFLD - ALLOW EDITING OF ESP FIELD

TBX$ALWFLD reverses the effect of the TBX$SKPFLD subroutine. TBX$SKPFLD leaves a field on the screen but keeps the cursor from moving to it, so it can’t be edited. TBX$ALWFLD "un-skips" a field, allowing it to be edited again.

INPUTS

A1 = Pointer to screen.
D0 = Field number to allow.

FORMAT

```
TOOLBX TBX$ALWFLD
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Field not found.

TBX$CLRFLS - CLEAR A BLOCK OF FIELDS

TBX$CLRFLS clears the field buffers of a block of ESP screen fields.

INPUTS

A1 = Pointer to screen.
D6 = Field number of first field to start clearing.
D7 = Number of fields to clear.

FORMAT

```
TOOLBX TBX$CLRFLS,SJT
```

OUTPUTS

None.

TBX$CLSSCR - CLOSE ESP SCREEN

TBX$CLSSCR closes an ESP screen previously opened by TBX$OPNSCR. It clears the window area for the screen. If there are any other ESP screens or windows open, the cleared window is updated with whatever was previously displayed in that area.

INPUTS

A1 = Pointer to screen.

FORMAT

```
TOOLBX TBX$CLSSCR
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Error, D6 contains error code.

TBX$DSPERR - DISPLAY MESSAGE TEXT

TBX$DSPERR displays the specified message text, centered on the bottom line of the terminal. It waits for the user to press a key before continuing.

TBX$DSPERR normally rings the terminal bell and the part of the message provided by the application blinks to draw the user's attention. You can change these options, as described below.

TBX$DSPERR sets the N-flag to 1 if the user presses MENU to acknowledge the message. Your application can then abort the current access if appropriate.

INPUTS

A6 = Message text,
D6 = Option flags. It can contain any combination of:

DE%NAK	User does not need to press a key to acknowledge.
DE%NBL	Do not blink message.
DE%NDG	Do not ring bell.
DE%NAM	Display `press any key to continue` as part of message.
DE%TTY	Display message at current cursor position. This is normally used for non-screen oriented applications.

FORMAT

```
TOOLBX TBX$DSPERR,SJT,Ax,PSH
```

OUTPUTS

Z-flag = 1: User acknowledged.
N-flag = 1: User pressed MENU.

TBX$DSPSCR - UPDATE ESP SCREEN DISPLAY

The TBX$DSPSCR subroutine updates an ESP screen display. It redisplays all changed fields.

INPUTS

A1 = Pointer to screen.
D2 = Display filler character. This is normally a space.

FORMAT

```
TOOLBX TBX$DSPSCR
```

OUTPUTS

Z-flag = 1: All fields are display only, D1 = MENU (XMENU for eight-bit characters).
Z-flag = 0: At least one field is editable, D1 is unchanged.

TBX$FETSCR - LOAD ESP SCREEN FROM DISK

The TBX$FETSCR subroutine fetches an ESP screen file from the disk and puts it in the specified buffer. The buffer may be less than 512 bytes, but must be large enough to hold the entire screen.

Before using TBX$FETSCR, you need to use TBX$SQFLSZ to size the file and TBX$MEMALC to allocate the memory necessary to load the screen. The DDB should be completely set up and contain the information normally returned by LOOKUP so the file size doesn't have to be redetermined.

TBX$FETSCR takes care of any load conversions and setup required. You should always use TBX$FETSCR to load an ESP screen.

You do not have to open the DDB before using TBX$FETSCR. The DDB is closed at the end of processing.

INPUTS

A1 = Where to load the screen.
A2 = Pointer to DDB of screen file to load.

FORMAT

```
TOOLBX TBX$FETSCR
```

OUTPUTS

A1 = Loaded screen.
A2 = Updated DDB.
Z-flag = 1: No errors.
Z-flag = 0: Error detected, D.ERR(A2) contains error code.

TBX$GETBNM - GET BINARY VALUE

The TBX$GETBNM subroutine gets a binary value (a four-byte integer) from an ESP screen field and returns it in register D1.

INPUTS

A1 = Pointer to screen.
D6 = Field number.

FORMAT

```
TOOLBX TBX$GETBNM,SJT
```

OUTPUTS

D1 = Value from field.

TBX$GETCHR - GET NEXT CHARACTER

The TBX$GETCHR subroutine gets the next available character from the terminal or an indirect input stream. It handles all standard desk accessory requests, time-out conditions, function key translation, etc.

INPUTS

None.

FORMAT

```
TOOLBX TBX$GETCHR
```

OUTPUTS

D1 = Character.

TBX$GETDAT - GET DATE VALUE

The TBX$GETDAT subroutine gets a date value in internal and separated format from an ESP screen field. The field can be either a date field or a text field.

INPUTS

A1 = Pointer to screen.
D6 = Field number.

FORMAT

```
TOOLBX TBX$GETDAT,SJT
```

OUTPUTS

D6 = Date in separated format.
D7 = Date in internal format.
Z-flag = 1: Valid date.
Z-flag = 0: Invalid date; D6, D7 = 0.

TBX$GETFV6 - GET FLOATING POINT VALUE

The TBX$GETFV6 subroutine gets a six-byte floating point value from an ESP screen field and returns it in the specified buffer.

INPUTS

A1 = Pointer to screen.
A6 = Buffer for floating point number.
D6 = Field number.

FORMAT

```
TOOLBX TBX$GETFV6,SJT,Ax,PSH
```

OUTPUTS

A6 = Pointer to floating point number.

TBX$GETPUI - GET POP-UP ITEM NUMBER

The TBX$GETPUI subroutine gets the pop-up menu item number from the specified screen field. Item numbers start at one, not zero.

INPUTS

A1 = Pointer to screen.
D6 = Field number.

FORMAT

```
TOOLBX TBX$GETPUI,SJT
```

OUTPUTS

D1 = Pop-up item number.
Z-flag = 0 if field is not a pop-up field.

TBX$GETSCR - PROCESS ESP SCREEN

The TBX$GETSCR subroutine is the heart of the ESP user interface. It performs all screen display updating and manages all user entries.

Processing remains in TBX$GETSCR until a pre-edit or post-edit interrupt occurs or the user presses a key that isn't understood by TBX$GETSCR (see Chapter 5).

TBX$GETSCR handles all standard desk accessory requests, time-out conditions, function key translation, etc. See the "Checking Outputs" section, below.

INPUTS

A1 = Pointer to screen.
D0 = Field number to start editing on.
D1 = Entry edit character.
D2 = Display filler character. This is normally a space.

FORMAT

```
TOOLBX TBX$GETSCR
```

OUTPUTS

A1 = Pointer to screen.
D0 = Field number of last field edited.
D1 = Character that caused GETSCR to return.

CHECKING THE OUTPUT

When TBX$GETSCR returns to your application, you should check D1 to see what character caused the return. The following special symbols are defined and available from the toolbox:

$SPACE	ASCII value 32 decimal
$MENU	MENU key value (seven-bit)
$EXEC	EXECUTE key value (seven-bit)
$XMENU	MENU key value (eight-bit)
$XEXEC	EXECUTE key value (eight-bit)
FL%IBE	Pre-edit interrupt flag
FL%IAE	Post-edit interrupt flag

One check you should always perform is for the MENU key. This is usually a user request to leave the current level of the application. Also, unless the timeout character has been redefined, this could be a MENU caused by a timeout. The correct way to check for MENU is:

```
CMPB    D1, #$MENU        ; Was MENU key pressed?
BEQ     ALLDUN            ; Yes, time to leave.
```

If you use eight-bit characters, check for $XMENU instead of $MENU.

If you are expecting any pre-edit or post-edit interrupts, you can check for them using this procedure:

```
BTST    #FL%IBE, D1       ; Interrupt before edit?
BNE     PREEDIT.INTERRUPT; Yes, process it. D0 has field #.

BTST    #FL%IAE, D1       ; Interrupt after edit?
BNE     POSTEDIT.INTERRUPT; Yes, process. D0 has field #.
```

You can use the CHRCHK macro to check for a specific character in either seven-bit or eight-bit mode. See Chapter 5.

You can now check for special characters. As an example, here is a test for the EXECUTE key, using the symbol for eight-bit characters:

```
CMPB    D1,#$XEXEC        ; EXECUTE key pressed (8-bit)
BEQ     PROCESS.RECORD  ; Yes; execute appropriately.
```

You can use similar code to check for any special characters, such as other function keys, defined by your application. If the character causing the return is meaningless to both ESP and your application, it is probably best to ignore it, since an error message could be distracting or confusing. You can branch back to the TBX$GETSCR call and allow the user to continue.

Before branching back to TBX$GETSCR, set D1 to an allowable character (normally a CTRL / U). If you leave the meaningless character in D1, TBX$GETSCR will immediately return to your program, and go into a loop.

TBX$GETSTR - GET STRING VALUE

The TBX$GETSTR subroutine gets a string value from an ESP field and places it in the specified string area. If the area is smaller than the string, the string is truncated. If the area is larger than the string field, it is padded with nulls.

INPUTS

A1 = Pointer to screen.
A6 = Buffer to return string into.
D6 = Field number.
D7 = Number of bytes to return. If -1, use maximum field size.

FORMAT

```
TOOLBX TBX$GETSTR,SJT,Ax,PSH
```

OUTPUTS

A6 = Pointer to buffer with string.
D7 = Number of actual bytes returned.
Z-flag = 1 if D7 is 0.

TBX$GETYNF - GET YES/NO VALUE

The TBX$GETYNF subroutine gets a yes/no (or true/false) value from an ESP screen field.

INPUTS

A1 = Pointer to screen.
D6 = Field number.

FORMAT

```
TOOLBX TBX$GETYNF,SJT
```

OUTPUTS

D7 = -1 for Yes, 0 for No.
Z-flag = 1: Yes.
Z-flag = 0: No.

TBX$GTUSER - GET USER PARAMETERS

The TBX$GTUSER subroutine returns various user specific parameters. These parameters are stored in the user impure area of TOOLBX.SYS. They are used by ESP applications; some are set by SIGNIN when the user logs in.

INPUTS

A2 = Buffer for string or floating point values.
D1 = Return buffer size.
D0 = Option number (see list below).

FORMAT

```
TOOLBX TBX$GTUSER
```

OUTPUTS

A2 = Pointer to buffer containing string or floating point value.
D1 = Returned value for integer option.
Z-flag = 1: Successful.
Z-flag = 0: Error (probably invalid option number).
N-flag = 1: Could not locate reserved area for this user.

OPTIONS

The following option number symbols are defined. For more information on many of these parameters, see the section on the GETUSR XCALL in Chapter 4.

SYMBOL	TYPE	DESCRIPTION
GU$FLG	Integer	SIGNIN user flags
GU$RDS	Integer	User read security
GU$WTS	Integer	User write security
GU$DFP	String(6)	Default printer name
GU$TMO	Integer	Timeout period in seconds
GU$TOC	Integer	Timeout character ASCII value
GU$NAM	String(10)	User name
GU$BCL	Integer	Default background color
GU$FCL	Integer	Default foreground color
GU$CMP	Integer	Company number
GU$MNU	Integer	SIGNIN menu item number
GU$CNM	String(60)	Company name
GU$HLP	String(50)	Default help file
GU$IIN	Integer	Indirect input active flag (0=No)
GU$LVL	Integer	User level from MUSER
GU$EXP	Integer	User expertise from MUSER
GU$IIL	Integer	Number of active indirect input levels
GU$TRM	String(6)	Terminal driver name
GU$JOB	String(10)	Job name
GU$RDV	String(10)	User's root device
GU$RPN	String(10)	User's root account
GU$LKS	Integer	LOKSER active indicator (0=No)
GU$LDV	String(10)	Current device
GU$LPN	String(10)	Current account
GU$JNM	Integer	Job number (in job table)
GU$ACM	Floating	Accumulator value (add desired accumulator number to GU$ACM)

TBX$HIDFLD - HIDE ESP FIELD

The TBX$HIDFLD subroutine hides a field on an ESP screen. The field becomes invisible and therefore can't be edited.

You can reverse the effect of TBX$HIDFLD by using TBX$SHWFLD.

INPUTS

A1 = Pointer to screen.
D0 = Field number to hide.

FORMAT

```
TOOLBX TBX$HIDFLD
```

OUTPUTS

Z-flag = 1: Field found and hidden.
Z-flag = 0: Field not found.

TBX$MEMALC - ALLOCATE MEMORY PACKET

The TBX$MEMALC subroutine allocates a memory packet within the memory pool.

If you are going to use TBX$FETSCR to load a file into this memory packet, you may want to use TBX$SQFLSZ to determine the size of the file so you know how big a packet to allocate.

INPUTS

A6 = Base of memory pool.
D6 = Size of desired memory packet, in bytes.

FORMAT

```
TOOLBX TBX$MEMALC,SJT,Ax,PSH
```

OUTPUTS

A6 = Allocated memory packet.
Z-flag = 1: Successful.
V-flag = 1: Insufficient memory available.

TBX$MEMCHG - CHANGE SIZE OF MEMORY PACKET

The TBX$MEMCHG subroutine changes the size of a memory packet you have previously allocated using TBX$MEMALC. If you want to make the packet larger, it is resized if possible. If this isn't possible, a new packet is allocated and the contents of the old one are copied into it.

If the new packet size is smaller than the old, the packet is resized and the excess memory is returned to the memory pool.

INPUTS

A6 = Base of memory pool.
D6 = Pointer to memory packet to resize.
D7 = Desired new packet size.

FORMAT

```
TOOLBX TBX$MEMCHG,SJT,Ax,PSH
```

OUTPUTS

D6 = Memory packet.

The packet may have been moved to accommodate resizing. Be sure to point to the new location for any further use of this packet.

Z-flag = 1: Successful.
Z-flag = 0: Failure.
V-flag = 1: Insufficient memory to perform resizing.

TBX$MEMDEA - DEALLOCATE MEMORY PACKET

The TBX$MEMDEA subroutine deallocates a memory packet you have previously allocated using TBX$MEMALC. It returns the space to the memory pool. If the specified packet is not from the memory pool, this subroutine has no effect.

INPUTS

A6 = Base of memory pool.
D6 = Address of memory packet to delete.

FORMAT

```
TOOLBX TBX$MEMDEA,SJT,Ax,PSH
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Error detected. Probably packet address was out of range.

TBX$MEMFRE - GET MEMORY POOL FREE SIZE

The TBX$MEMFRE subroutine returns the total amount of free memory in the memory pool, and the size of the largest free packet in the pool.

INPUTS

A6 = Base of memory pool.

FORMAT

```
TOOLBX TBX$MEMFRE,SJT,Ax,PSH
```

OUTPUTS

D6 = Number of free bytes available.
D7 = Size of largest contiguous packet available.

TBX$MEMINI - INITIALIZE MEMORY POOL

The TBX$MEMINI subroutine initializes an area of memory for use by the memory manager. This area is known as the memory pool. All other calls to the memory manager require the base address of the memory pool.

You must use TBX$MEMINI before using any of the other memory management subroutines.

INPUTS

A6 = Base of memory pool.
D6 = Size of memory pool in bytes.

FORMAT

```
TOOLBX TBX$MEMINI,SJT,Ax,PSH
```

OUTPUTS

Z-flag = 1: Successful.
V-flag = 1: Insufficient memory to initialize the memory pool.

TBX$MPLALC - ALLOCATE PACKET IN SHARED MEMORY POOL

The TBX$MPLALC subroutine allocates a memory packet in the TOOLBX.SYS central memory pool.

This memory pool is shared by all toolbox users. You must return all resources when finished.

Since this area is shared, you should use it only when you can’t allocate the resources any other way.

INPUTS

D6 = Size of desired packet.

FORMAT

```
TOOLBX TBX$MPLALC,SJT
```

OUTPUTS

A6 = Allocated memory packet.
Z-flag = 1: Successful.
V-flag = 1: Insufficient memory available.

TBX$MPLDEA - DEALLOCATE PACKET IN SHARED MEMORY POOL

The TBX$MPLDEA subroutine deallocates a memory packet you've previously allocated in the shared TOOLBX.SYS memory pool using TBX$MPLALC.

Because the shared memory pool is used by all toolbox users, be absolutely certain to return all resources using this subroutine when you are finished.

INPUTS

D6 = Address of memory packet to delete.

FORMAT

```
TOOLBX TBX$MPLDEA,SJT
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Error detected. Probably packet address was out of range.

TBX$MPLFRE - FIND FREE AREA OF SHARED MEMORY POOL

The TBX$MPLFRE subroutine finds the amount of free memory and the size of the largest available memory packet in the shared TOOLBX.SYS memory pool.

The TOOLBX.SYS memory pool is shared among all toolbox users. You should use it only when necessary, and be sure to return all resources when finished.

INPUTS

None.

FORMAT

```
TOOLBX TBX$MPLFRE
```

OUTPUTS

D6 = Number of free bytes available.
D7 = Size of largest contiguous packet available.

TBX$OPNSCR - OPEN ESP SCREEN

The TBX$OPNSCR subroutine opens an ESP screen and prepares it for processing. If another ESP screen or window is currently open, the window area for the new screen is cleared.

You need to use TBX$FETSCR to fetch the screen file from disk before using this subroutine. TBX$OPNSCR does not display the screen—use TBX$GETSCR to do so.

INPUTS

A1 = Pointer to screen.

FORMAT

```
TOOLBX TBX$OPNSCR
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Error detected, D6 contains error code.

TBX$SCRPRT - PRINT ESP SCREEN

The TBX$SCRPRT subroutine prints an ESP screen. It is normally used with report screens. You can choose which field classes to print.

INPUTS

A1 = Pointer to screen.
A2 = DDB of file to output screen to. DDB must already be open.
A4 = Buffer used to "unpack" the screen. Must be larger than entire screen (rows multiplied by columns).
D0 = Screen print flags. It can contain any combination of the following:

PS%SPL	Spool the file after closing.
PS%BRD	Print border lines around the image.
PS%RUL	Print a ruler at the top and side of the image.
PS%CLS	Close the DDB after printing the image.
PS%NPR	Do not print; unpack only.

D2 = Filler character and field class(es) to print. It can contain any combination of the following:

FC$TTL	Print all title fields.
FC$HED	Print all detail header fields.
FC$DET	Print all detail fields.
FC$ST1	Print all level one subtotal fields.
FC$ST2	Print all level two subtotal fields.
FC$ST3	Print all level three subtotal fields.
FC$GTL	Print all grand total fields.

FORMAT

```
TOOLBX TBX$SCRPRT
```

OUTPUTS

D6 = Number of lines printed.
Z-flag = 1: Successful.
Z-flag = 0: Error detected, D6 contains error code.
V-flag = 1: Insufficient memory.

TBX$SETBNM - SET BINARY VALUE

The TBX$SETBNM subroutine sets a binary value (a four-byte integer) in register D1 into an ESP screen field.

This routine does not update the screen display. To display the new field value, use TBX$GETSCR or TBX$DSPSCR to redisplay the screen.

INPUTS

A1 = Pointer to screen.
D6 = Field number.
D1 = Value to load.

FORMAT

```
TOOLBX TBX$SETBNM,SJT
```

OUTPUTS

None.

TBX$SETFV6 - SET FLOATING POINT VALUE

The TBX$SETFV6 subroutine sets a six-byte floating point format value from a specified buffer into an ESP screen field.

This routine does not update the screen display. To display the new field value, use TBX$GETSCR or TBX$DSPSCR to redisplay the screen.

INPUTS

A1 = Pointer to screen.
A6 = Pointer to floating point number to load.
D6 = Field number.

FORMAT

```
TOOLBX TBX$SETFV6,SJT,Ax,PSH
```

OUTPUTS

None.

TBX$SETPUI - SELECT POP-UP ITEM

The TBX$SETPUI subroutine selects the specified item from the pop-up menu item list for the specified screen field. Pop-up item numbers start at one, not zero.

This routine does not update the screen display. To display the new field value, use TBX$GETSCR or TBX$DSPSCR to redisplay the screen.

INPUTS

A1 = Pointer to screen.
D1 = Pop-up menu item number.
D6 = Field number.

FORMAT

```
TOOLBX TBX$SETPUI,SJT
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Error. Possible wrong field type or out of range item number.

TBX$SETSTR - SET STRING VALUE

The TBX$SETSTR subroutine takes a string value from the specified string area and places it in an ESP field. If the field is smaller than the area, the string is truncated. The string value length is up to the specified size or until terminated by a null.

This routine does not update the screen display. To display the new field value, use TBX$GETSCR or TBX$DSPSCR to redisplay the screen.

INPUTS

A1 = Pointer to screen.
A6 = Pointer to string to load.
D6 = Field number.
D7 = Size of string in bytes (0 clears the field).

FORMAT

```
TOOLBX TBX$SETSTR,SJT,Ax,PSH
```

OUTPUTS

D7 = Actual number of bytes loaded.

TBX$SETYNF - SET YES/NO VALUE

The TBX$SETYNF subroutine sets a yes/no (or true/false) value from register D7 into an ESP screen field.

This routine does not update the screen display. To display the new field value, use TBX$GETSCR or TBX$DSPSCR to redisplay the screen.

INPUTS

A1 = Pointer to screen.
D6 = Field number.
D7 = Non-zero for yes, zero for no.

FORMAT

```
TOOLBX TBX$SETYNF,SJT
```

OUTPUTS

None.

TBX$SHWFLD - SHOW ESP FIELD

The TBX$SHWFLD subroutine "unhides" a field on an ESP screen. Unless read secured, the field displays and can be edited if it is otherwise editable.

TBX$SHWFLD reverses the effect of TBX$HIDFLD.

INPUTS

A1 = Pointer to screen.
D0 = Field number to show.

FORMAT

```
TOOLBX TBX$SHWFLD
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Field not found.

TBX$SKPFLD - SKIP ESP FIELD

The TBX$SKPFLD causes a field on an ESP screen to be skipped. The field can’t be edited, but stays visible, unless it is read secured or hidden. You can reverse the effect of a TBX$SKPFLD with TBX$ALWFLD.

INPUTS

A1 = Pointer to screen.
D0 = Field number.

FORMAT

```
TOOLBX  TBX$SKPFLD
```

OUTPUTS

Z-flag = 1: Successful.
Z-flag = 0: Field not found.

TBX$SQFLSZ - SEQUENTIAL FILE SIZE

The TBX$SQFLSZ subroutine gets the size, in bytes, of a sequential file. A LOOKUP call is performed to get the file size.

You can use this call to determine a screen file size before allocating a memory packet (using TBX$MEMALC) to hold the screen file. However, remember that FETCH always requires at least a 512 byte fetch buffer and special processing is required to load into a smaller area. You should always use TBX$FETSCR instead of FETCH to fetch an ESP screen file.

INPUTS

A2 = DDB for the file to be sized. Do NOT open it first.

FORMAT

```
TOOLBX  TBX$SQFLSZ
```

or:

```
TOOLBX  TBX$SQFSZ1
```
(see below)

OUTPUTS

A2 = Updated DDB.
D6 = Size of file in bytes.
Z-flag = 1: Successful.
N-flag = 1: Error. Error code is in D.ERR(A2).

TBX$SQFSZ1

If the file DDB has already had a LOOKUP operation performed, it is more efficient to use TBX$SQFSZ1 than TBX$SQFLSZ. The two routines are identical, except that TBX$SQFSZ1 does not perform the LOOKUP.

TBX$STUSER - SET USER PARAMETERS

The TBX$STUSER subroutine sets various user specific parameters. These parameters are stored in the user impure area of TOOLBX.SYS. They are used by ESP applications; some are set by SIGNIN when the user logs in.

INPUTS

D0 = Option number (see list below).

Additional input depends on the type of value required for the option you're setting.

If the option takes a string argument:

A2 = String to set.
D1 = Size of string.

If the option takes an integer argument:

D1 = Value of argument.

If the option is an accumulator:

A2 = Floating point value to set.

FORMAT

```
TOOLBX TBX$STUSER
```

OUTPUTS

Z-flag = 1: Successful.
N-flag = 1: Could not locate user impure area.
V-flag = 1: Error code in D6.

OPTIONS

The following option number symbols are defined. For more information on many of these parameters, see the section on the SETUSR XCALL in Chapter 4.

SYMBOL	TYPE	DESCRIPTION
SU$FLG	Integer	SIGNIN user flags
SU$RDS	Integer	User read security
SU$WTS	Integer	User write security
SU$DFP	String(6)	Default printer name
SU$TMO	Integer	Timeout period in seconds
SU$TOC	Integer	Timeout character ASCII value
SU$NAM	String(10)	User name
SU$BCL	Integer	Default background color
SU$FCL	Integer	Default foreground color
SU$CMP	Integer	Company number
SU$MNU	Integer	SIGNIN menu item number
SU$CNM	String(60)	Company name
SU$HLP	String(50)	Default help file
SU$IIF	Integer	Indirect input file specification.
SU$LVL	Integer	User level from MUSER
SU$EXP	Integer	User expertise from MUSER
SU$IIS	String(D1)	Indirect input string
SU$CII	Integer	Clear indirect input levels.
SU$III	Integer	Indirect input iteration count.
SU$KRF	String(50)	File specification of keystroke record file.
SU$CKR	Integer	Close keystroke record levels.
SU$ACM	Floating	Accumulator value (add desired accumulator number to SU$ACM)

TBX$TRMCAP - GET TERMINAL CAPABILITIES

The TBX$TRMCAP subroutine returns the terminal feature flags and the number of available rows and columns. Terminal feature flags are documented in the *AMOS Terminal Service System User's Manual*.

INPUTS

None.

FORMAT

```
TOOLBX TBX$TRMCAP
```

OUTPUTS

D6 = Width of terminal in characters.
D7 = Terminal capabilities flags.
A6 = Size of terminal in rows.

TBX$TRMINI - INITIALIZE TERMINAL SESSION

The TBX$TRMINI subroutine initializes a terminal session. It can put the terminal into data mode entry, set up the terminal status lines, if any, clear local accumulators, set up printer definitions, and clear the terminal screen.

INPUTS

A1 = Optional top status line message (0 if no text). See below for format.
A2 = Optional bottom status line message (0 if no text). See below for format.
D1 = Size of top status line message.
D2 = Size of bottom status line message.
D6 = Option flags. It can contain any combination of:

TI%NCS	Don’t clear the terminal screen.
TI%NCA	Don’t clear the local accumulators.
TI%NSL	Don’t change the status lines.
TI%NPD	Don’t read the printer definition file, PRTDEF.lan.
TI%DII	Discard one level of indirect input.
TI%RST	Reset the terminal status flags to non-data mode input.
TI%NEW	Set eight-bit character mode.

If you use eight-bit characters be sure to pass TI%NEW every time you use TBX$TRMINI.

FORMAT

```
TOOLBX TBX$TRMINI,SJT
```

OUTPUTS

None.

STATUS LINE FORMATS

The top status line displays the application name, followed by a dash, followed by the top status line message. This is followed by the user name or, if the name is not available, the current device and account.

The bottom status line displays the status line message text starting at the left margin. The current date and time are displayed to the right.

If you don’t want a top status line message, A1 should be 0; this clears the top status line completely. If A1 is not zero, but D1 is, then the normal status line information displays but the message text portion is blank.

If you don’t want a bottom status line message, A2 should be 0; this clears the bottom status line completely. If A2 is not zero, but D2 is, then the date and time display but the message text portion is blank.

CHAPTER FOUR
AlphaBASIC XCALLS

This chapter describes the AlphaBASIC XCALLs included with ESP. These XCALLs let you call the assembly language routines included in the TOOLBX.SYS file from your AlphaBASIC application.

The XCALLs are listed in alphabetical order.

All ESP AlphaBASIC XCALL routines are reentrant and reusable. You do not have to pre-load any of them into system or user memory, but you can do so to reduce load time and memory requirements.

This chapter assumes you already know how to program in AlphaBASIC, and how to use XCALLs. Refer to the *AlphaBASIC User's Manual* or the *AlphaBASIC XCALL User's Manual* for more information.

ALERT - DISPLAY ERROR MESSAGE

The ALERT XCALL displays the specified error message on the bottom line of the terminal or sends it to a selected buffer. The format for ALERT is:

```
XCALL ALERT,msg'number{,destination}
```

msg'number is the number of the message. Messages are defined in the TBXMSG.BSI file. See Chapter 6 for how you can modify the messages in this file.

destination is an optional buffer to output the message to. If you include a destination, the message is not displayed on the screen.

When the message is displayed on the terminal, the user must press a key to clear it and continue. If there is room on the line, a prompt saying this is added to the error message.

BUSY - ACCESS BUSY INDICATOR

The BUSY XCALL subroutine gives you access to ESP's busy indicators. There are three different types of indicators, all displayed on the bottom status line: The three types are:

1. The blinking word BUSY.
2. A percentage of completion.
3. A number-complete-of-total in the format n / x.

You can use the last two types only when you know the total count of the operation and can measure how far along the process is—for example, searching through a data file when you know how many records there are in it. If you are performing an operation of unknown length, such as reading a file you don't know the size of, use the simple busy indicator.

The calling format for BUSY is:

```
XCALL BUSY, option, mode, {count}
```

option is the type of indicator you want to use. There are three values defined:

BZY'TX	Flashing BUSY
BZY'PC	Percentage of completion
BZY'NM	Number-complete-of-total

mode is the type of operation you want to perform. The different modes are:

BZY'OFF	Initialize or reset the busy indicator (use with BZY'PC and BZY'NM only)
BZY'TGL	Toggle or advance the indicator.

count is only used with BZY'OFF. When you initialize the indicator, you should set this to the "complete" number; when the process has finished, set count to zero to clear the indicator.

For a simple flashing BUSY, you just need to put this line somewhere in your operation loop:

```
XCALL BUSY,BZY'TX,BZY'TGL
```

To use either of the progressive indicators, you need to call BUSY originally using the option you want with BZY'OFF set and the total count for the operation. For example:

```
XCALL BUSY,BZY'PC,BZY'OFF,1000
```

Then, to update the indicator, call BUSY again using the BZY'TGL mode. Each time you call BUSY in this mode, BUSY advances its counter one toward the total you have set, and displays the new count or percentage. So, make sure you call BUSY at the appropriate times to advance the counter.

Finally, when the operation is complete, call BUSY again using BZY'OFF with the count set to zero. For example:

```
XCALL BUSY,BZY'PC,BZY'OFF,0
```

CLRBLK - CLEAR BLOCK ON TERMINAL SCREEN

CLRBLK clears a rectangular area of the user's terminal screen. You would normally use this to erase a windowed ESP screen which overlays a non-ESP screen display.

The calling format for CLRBLK is:

```
XCALL CLRBLK,row,col,height,width
```

row is the top row of the area to clear. ***col*** is the leftmost column of the area to clear. ***height*** is the number of rows to clear; ***width*** is the number of columns to clear.

CLRFLS - CLEAR FIELDS ON ESP SCREEN

CLRFLS clears one or more fields on an ESP screen. The fields must be consecutively numbered. The screen must already have been FETCHed into an X formatted variable. The calling format for CLRFLS is:

```
XCALL CLRFLS,screen,start,number
```

screen is the variable containing the screen.

start is the number of the first field to clear.

number is the number of fields to clear.

CLSSCR - CLOSE ESP SCREEN

The CLSSCR XCALL removes a screen from the list of currently open screens. ESP maintains this list to ensure proper windowing of screens on the terminal. You should always use CLSSCR to remove a screen from the terminal.

To call the CLSSCR subroutine, use this syntax:

```
XCALL CLSSCR, screen
```

screen is an X format variable containing the ESP screen.

ERRDSP - DISPLAY ERROR MESSAGE

The ERRDSP XCALL displays an error message on the last line of the terminal and rings the terminal bell. The message is centered on the line and blinking. If the length of the message allows, this phrase also appears:

```
...press any key to continue
```

The message remains until the user acknowledges it by pressing a key. It then clears and control returns to the calling program.

To call the ERRDSP subroutine, use this syntax:

```
XCALL ERRDSP, err'txt$ {,flags}
```

err'txt$ is a string variable or expression containing the text to display. The text may be no longer than the width of the screen.

flags can be any combination of the following:

ERRDSP'NAK	Do not require the user to press a key to acknowledge the message.
ERRDSP'NBL	Do not blink the message.
ERRDSP'NDG	Do not ring the terminal bell.
ERRDSP'TTY	Display the message at the current cursor position; do not use any terminal features.

FETCH - LOAD SCREEN DEFINITION FILE

FETCH loads screens generated by SEDIT into an AlphaBASIC X format variable for use with the ESP screen XCALL subroutines, such as GTSCR, GETVAL, and SETVAL.

To call FETCH, use this syntax:

```
XCALL FETCH, file'spec$, load'buffer, ret'code
```

file'spec$ is a string variable or expression containing a standard AMOS file specification.

If you don't include an extension in file'spec$, FETCH assumes the extension for the current language. If it can't find a file with the language extension, it looks for one with the extension .SCR.

If you don't include an account or device, FETCH follows the standard library search path:

1. The current account
2. The [,0] account for this project
3. DSK0:[7,6] for AlphaBASIC or
4. DSK0:[7,35] for AlphaBASIC Plus

If FETCH cannot find the file or the AlphaBASIC mapped variable is smaller than the size of the file, an error code is returned in the specified return variable.

load'buffer is a mapped variable into which the specified file is loaded. load'buffer must:

- Be mapped as X format.
- Be at least 512 bytes long.
- Be defined as a MAP1 variable. This ensures it starts on an even boundary.

ret'code is a floating point variable in which the return code from the operation is returned. The codes are:

CODE	MEANING
0	Successful load.
2	Insufficient memory to load file; you should increase the size of the load'buffer$ variable.
3	The specified file was not found.

If an error code other than these occurs, see the error code list in the *AMOS Monitor Calls Manual*.

FILINB - READ ONE BYTE FROM A SEQUENTIAL FILE

The FILINB XCALL subroutine reads the next available byte from a sequential file into a variable. The file must already have been opened for input. The calling format for FILINB is:

```
XCALL FILINB,channel#,ret'variable
```

channel# is the channel number of the open file to read the byte from.

ret'variable is the variable to read the byte into.

GETFSC - GET FIELD SECURITY CLEARANCE STATUS

You can use the GETFSC XCALL to determine the security clearance status of a field in a screen created using ESP. GETFSC loads the values reflecting the user security status of the specified field into the specified variables.

To call the GETFSC subroutine, use this syntax:

```
XCALL GETFSC,screen,field'number,var1{,var2,var3,var4}
```

screen is an X format mapped variable containing an ESP screen, previously loaded using ESP's FETCH XCALL.

field'number is a floating point variable or expression specifying the field number whose security status you want.

var1 is a variable whose value is set to either 0 or -1, depending on the result of the read security test as described below.

var2 is an optional variable whose value is set to either 0 or -1 depending on the result of the write security test as described below.

var3 is an optional variable whose value is set to the field's actual read security level.

var4 is an optional variable whose value is set to the field's actual write security level.

Read Security Test

If the user's read security as defined in the user impure area is at or above the read security of the field in question, then var1 is returned as -1. Otherwise, var1 is returned as 0. For more information about the user impure area, see the section on SETUSR in this chapter.

Write Security Test

If the user's write security as defined in the user impure area is at or above the write security of the field in question, then var2 is returned as -1. Otherwise, var2 is returned as 0. For more information about the user impure area, see the section on SETUSR in this chapter.

GETPUI - GET POP-UP ITEM

The GETPUI XCALL retrieves the current value of a pop-up item field. Its format is:

```
XCALL GETPUI,screen,field,value
```

screen is the unformatted variable holding the screen file.

field is the number of the field you want the pop-up item value from. If a variable, it can be either floating point or integer format.

The number of the currently chosen pop-up item is returned in ***value***, which should be a floating point variable.

GETPUI returns the number of the item chosen, not the item name. For example, if the pop-up items for a color field are green, blue, and red, in that order, green equals 1, and so on.

Since a number is returned, not a string, you can use a simple ON GOSUB or ON GOTO command to perform the action appropriate for each choice rather than having to do multiple string comparisons to see which selection was made.

GETUSR - GET USER IMPURE AREA ELEMENTS

The GETUSR XCALL subroutine gets a specified value from the user impure area in TOOLBX.SYS. The user impure area is set up when TOOLBX.SYS is loaded into system memory during booting. You can set elements of the user impure area using the SETUSR XCALL, described later in this chapter.

To call the GETUSR subroutine, use this syntax:

```
XCALL GETUSR, element'number, var
```

element'number is a floating point variable or expression specifying which element of the user impure area to access.

The active element numbers are 1 through 19, 1000 through 1010, and 10000 through 10511. They are described below.

var is a variable or expression whose value is loaded from element'number.

For element numbers 1, 2, 3, 5, 6, 8, 9, 10, and 11, var must be a floating point variable.

For element number 4, var must be a string of at least 6 bytes.

For element number 7, var must be a string of at least 10 bytes.

For elements 12, 13, and 14, var must be a string variable.

GETUSR Elements

The following list describes all the active GETUSR element numbers:

- Element 1 is a decimal number indicating which of the following flags are set:

VALUE	INDICATES
1	User is on a modem; do not display on-line help messages.
2	Do not spool ESP screens; print to disk file only.
4	Use eight-bit characters.
32768	Allow CTRL / C during SIGNIN.

  The value for element 1 is the sum of all the flags that are set.

- Element 2 is the user's read security level. It can be from 0 through 254.
- Element 3 is the user's write security level. It can be from 0 through 254.

Remember: the higher the value, the greater the user's security clearance.

- Element 4 returns the user's current default printer name. A null string indicates the system default printer.

- Element 5 returns values from 0 to 65535. This is the application timeout period in seconds. A value of 0 means the application never times out.

- Element 6 is the ASCII character sent when the application times out. It can be from from 0 to 255. The most likely value is 155, which represents the [MENU] key. However, any other character is valid and may be used. Chapter 5 discusses ESP's handling of special keys.

- Element 7 contains the user's name, which may be any combination of printable characters up to 10 characters long. This element may be blank.

- Elements 8 and 9 are for color control on terminals supporting color. They may return values from 0 to 255. Element 8 is the user's default background color; element 9 is the user's default foreground color. The color selections for foreground and background follow Alpha Micro's standards.

  The standard color definitions and values are (these may be changed on some color terminals):

VALUE	COLOR
0	Black
1	White
2	Blue
3	Magenta
4	Red
5	Yellow
6	Green
7	Cyan

  Colors 8 through 15 are the dim versions of these colors.

- Element 10 is the currently selected company number. This value is normally maintained by SIGNIN, if active.

- Element 11 is the last menu selection made by the user from the SIGNIN menu. This value is normally maintained by SIGNIN, if active.

- Element 12 is the current company name, which may be any combination of printable characters up to 60 characters in length. This value is normally maintained by SIGNIN, if active. This item is useful for printing on reports and for displaying on the bottom status line.

- Element 13 is the current default help file specification.

- Element 14 is a string containing the current indirect input flags. It is the same as element 17.

Indirect input options are discussed in the SETUSR section of this chapter.

- Element 15 is the user's level, from 0 through 100.
- Element 16 is the user's expertise, from 0 through 100.
- Element 17 is a string containing the current indirect input flags. It is the same as element 14.
- Element 18 is the number of the current indirect input level.
- Element 19 is the current page number for reports. It is meaningful only if you are using the DART data base manager.
- Element 1000 is the name of the terminal driver for the user's job.
- Element 1001 is the user's job name.
- Element 1002 is the user's root device, as taken from MUSER.
- Element 1003 is the user's root account number, as taken from MUSER.
- Element 1004 is a flag indicating whether LOKSER is active or not.
- Element 1005 is the current device.
- Element 1006 is the current account.
- Element 1007 is the user's job number—the position in the job table.
- Element 1008 is the current debug level.
- Element 1009 is a flag indicating whether SIGNIN is active; if it is, the value is 1, if not, the value is 0.
- Element 1010 is the account specification for the SIGNIN: ersatz device. If there is no SIGNIN: ersatz device, it defaults to DSK0:[1,2].
- Elements 10000 through 10511 are floating point values corresponding to ESP expression accumulators @0 through @511. See the *SEDIT User's Guide* for information on accumulators.

GETVAL - GET VALUES FROM SCREEN FIELDS

The GETVAL XCALL subroutine gets a value or values from a field or fields in an ESP screen. You use GETVAL to retrieve one or more specific field values from the variable containing the entire screen file.

GETVAL loads a specified set of AlphaBASIC variables with the contents of fields from the ESP screen buffer specified in the calling sequence. It performs any necessary type conversions.

You must use FETCH to load the screen into a variable before using GETVAL. You would normally also use GTSCR to get screen input from the user before using GETVAL to extract specific values from the screen.

For date format fields, the string variable specified must be at least 6 characters long. If the string variable is less than 8 characters, the date is returned without slashes. If the string variable is 8 characters or more and the slashes field modifier is set, the date is returned with embedded slashes.

To call the GETVAL subroutine, use this syntax:

```
XCALL GETVAL,screen,field'number,var1{,var2,..., varn}
```

screen is an X format variable containing a screen created using ESP, previously loaded using ESP's FETCH XCALL.

field'number is a floating point variable or expression specifying the field number at which to start retrieving values from the specified ESP screen.

var1 is a variable whose value is set to the contents of field'number. It can be any AlphaBASIC variable type.

var2 is an optional variable whose value is set to the contents of the next sequential field after field'number.

Each succeeding variable is an optional variable whose value is set to the contents of the next field.

For example, if field'number is 5, var1 is loaded from field number 5, var2 is loaded from field number 6, and so on. This syntax is provided for convenience and may, but does not need to, replace multiple XCALL GETVAL statements.

GETYNF - GET YES/NO RESPONSE FROM FIELD

The GETYNF XCALL allows your program to get the results from a yes/no type field. You define a yes/no field in SEDIT by setting the field modifier for a text field to accept only a yes or no response. The only characters a user can enter in this type of field are the first letters of yes or or no as defined for the user's current language.

The format for calling GETYNF is:

```
XCALL GETYNF, screen,field'number,response
```

screen is an X format variable containing a screen created using ESP, previously loaded using ESP's FETCH XCALL.

field'number is a floating point variable or expression specifying the field number of the yes/no field.

response is a floating point, binary, or integer variable where the result is returned. Yes is returned as 1 and no is returned as 0.

GMENU - GENERALIZED MENU PROCESSING

The GMENU XCALL processes menus created by ESP. Once the menu screen has been FETCHed and opened (by OPNSCR), GMENU handles displaying the menu and accepting the user's selection. What GTSCR is to other input screens, GMENU is to menus.

In order for GMENU to work properly, the menu screen must follow certain conventions. How to define a menu screen to work with GMENU is described below.

The calling format for GMENU is:

```
XCALL GMENU,screen,char,field,select,num'sel{,first'sel{,type}
```

screen is an unformatted variable containing the screen file.

char is a floating point variable containing the first editing character to be processed by GMENU, and any special processing flags. When GMENU returns to the application, char contains the ASCII value of the character causing the return, in addition to the special processing flags.

See GTSCR for a description of the special processing flags and the checks you should perform on char after GMENU returns to the calling program.

field is a floating point variable containing the field number of the command line field (see the screen description, below). When GMENU returns to the application, field contains the number of the last field accessed.

select is a floating point variable to hold the user's menu selection. The first selection returns one, the second returns two, and so on. This number is not the same as the field number of the selection chosen unless the selection fields start with field number one.

num'sel defines the number of menu selection fields in the screen. It can be a floating point variable or expression.

first'sel is a floating point variable or expression containing the number of the first menu selection field. If you omit first'sel, the first selection field is considered to be one more than the value of field.

type contains an optional flag indicating the type of menu. 0 specifies a command menu; 1 indicates a numbered selection menu.

DESIGNING A MENU SCREEN TO USE WITH GMENU

In general terms, the screen file you use with GMENU must be a menu; that is, it should be a list of choices with one entry field where you enter the choice you want.

There are two ways for users to choose an option on a GMENU screen: type the name or number of the choice in the command line field; or use the cursor control keys to move to the choice they want and press [EXECUTE]. In either case, GMENU returns to the calling program; the number of the choice is returned in the select variable.

This is how to define the fields for a GMENU menu screen:

The *SEDIT User's Manual* describes how to set all the field contents and characteristics described below.

- One field is the command line field. This is the field defined by the field variable when you call GMENU. It should be the only field on the screen which allows an entry.

  When the user makes an entry in the command line field of a command type menu, GMENU compares it to the field buffer of each of the selection fields until it finds a match. The number of the matching selection is then returned to the calling program in the select variable.

  For a numbered menu, GMENU compares an entry in the command line to the numbers of the selection fields and returns a valid selection to the program in the select variable.

- The fields defining each menu choice must be consecutive. The number of the first field is given by the first'sel variable when you call GMENU; the number of choice fields is given by num'sel.

  For a command type menu, the field buffer of each choice field must contain the name of the operation to perform—this defines the entry the user needs to make for this selection. These fields should be non-changeable, but you should be able to move the cursor into them.

  You can set the number of characters required for a match for any particular choice by setting the field minimum size for the menu selection fields. For example, if two choices are edit and erase, if you set the minimum size for the edit field to one and the erase field to two, an entry of E will choose edit, and ER will be necessary to select erase.

  For numbered menus, GMENU determines the number of each choice from its position in the list of fields. Of course, the field buffer or field prompt should have the number of the selection so the user can see it.

  The on-line help message for each selection field should contain a description of the choice. These descriptions will appear when the cursor is in the field.

  To help the appearance of the menu screen, you may want to fill each selection field with blanks out to the maximum size setting and set the retain trailing blanks flag. This prevents the field fill underscore characters from appearing when the cursor moves into each field.

GTCHR - GET ONE CHARACTER FROM KEYBOARD

The GTCHR XCALL inputs non-field data. This subroutine lets an AlphaBASIC program can accept input from the screen when the information is not contained in a predefined ESP field.

A typical application for this subroutine would be to allow the user to press [NEXT SCREEN] ([CTRL] / [T]) to move forward through a report display.

GTCHR loads the specified AlphaBASIC variable with a single character input from the keyboard.

To call the GTCHR subroutine, use this syntax:

```
XCALL GTCHR, char{, flag}
```

char is a floating point variable into which the ISO Latin-I value of the entered character is loaded.

flag is an optional floating point variable or expression which, if set to -1, echoes the entered character on the screen. If this flag is not included or the value is 0, there is no echo of the character.

GTSCR - HANDLE SCREEN PROCESSING

The GTSCR XCALL subroutine activates ESP screen processing. This subroutine does ***all*** terminal input and output for an ESP screen and is the heart of the application program's screen/user interface.

GTSCR handles all screen oriented operations, such as:

- Insertion and deletion of characters.
- Field validation.
- Movement from field to field.
- Cursor movement within a field.

To call the GTSCR subroutine, use this syntax:

```
XCALL GTSCR, screen {, char, field'number}
```

screen is an X format mapped variable containing an ESP screen created by SEDIT, previously loaded using ESP's FETCH XCALL.

The screen should also have been opened using the OPNSCR XCALL before you use GETSCR.

char is a floating point variable containing the first edit character to be executed by GTSCR and any special processing flags.

When returning to the application program, char is set to the ASCII value of the character causing the return, plus any special processing flags (see below).

We suggest char be a different variable for each screen in your application. Since you can display and process two or more screens at once, if you use the same variable for char for multiple screens, you may write over the exit character or special processing flags for one screen with those from another.

The character returned by GTSCR resides in the low byte of char. Testing this character is discussed below.

field'number is a floating point variable specifying the field number where editing is to start. This variable is set to the then current field number upon return from GTSCR.

SPECIAL PROCESSING FLAGS

Special processing flags are defined in the following table. The flags are shown as decimal representations of binary digits. The first binary digit is represented in decimal as 1, the second binary digit as 2, the third as 4, and so on up to the sixteenth as 65536.

- 32768 (ESP'POSTEDIT)—when returned from GTSCR, indicates a post-edit interrupt has occurred. The field number where interrupt occurred is passed in field'number. The character causing the interrupt is returned in char.

  When passed to GTSCR, this indicates GTSCR should move to the next field, based on the edit character passed in the low byte of char. For example, if char contains the ↓ character (because that key was pressed to leave the field), GTSCR executes the ↓ operation before editing the next field.

- 16384 (ESP'PREEDIT)—when returned from GTSCR, indicates a pre-edit interrupt condition has occurred. The number of the field at which the interrupt occurred is passed in field'number. The character causing the interrupt is returned in char.

  When passed to GTSCR, this indicates GTSCR should let the user start editing the current field. This flag must be set upon entry into a field with pre-edit interrupt enabled to allow the user to actually edit the field.

- 8192 (ESP'REPAINT)—when returned from GTSCR, is not significant.

  When passed to GTSCR, this redisplays all fields on the screen, including boxes and display only fields. You should set this flag if the user program has destroyed the screen in any way, such as by a clear screen.

- 2048 (ESP'RESETCHANGE)—when returned from GTSCR, is not significant.

  When passed to GTSCR, this sets the current field to changed-value post-edit interrupt. This can be useful when an error has occurred in a post-edit interrupt filed, and you are returning the user to the field to make a correction.

- 1024 (ESP'DAF)—when returned from GTSCR, is not significant.

  When passed to GTSCR, this reevaluates all field expressions and redisplays all fields.

You can set each of these flags using the expression:

```
(CHAR OR flag'value)
```

flag'value can be

32768 or ESP'POSTEDIT to set post-edit interrupt.
16384 or ESP'PREEDIT to set for pre-edit interrupt.
8192 or ESP'REPAINT to set the redisplay flag.
2048 or ESP'RESETCHANGE to set changed-value post-edit interrupt.
1024 or ESP'DAF to cause reevaluation and redisplay.

You can test for each of the flags with the expression:

```
(CHAR AND flag'value)
```

The result is non-zero if the flag is enabled and zero if the flag is disabled.

For example, this statement tests for a post-edit interrupt:

```
IF CHAR AND ESP'POSTEDIT THEN ON FIELD CALL.......
```

RETURNING TO THE APPLICATION

GTSCR returns to the application only when one of the following occurs:

- The cursor enters a field which has the pre-edit interrupt field flag set.
- The cursor leaves a field which has the post-edit interrupt field flag set.
- The cursor leaves a field with the changed value post-edit field flag set after the user has changed the field.
- The user presses any non-printable key which is not defined to ESP. In general, this means most function keys or any control character or escape sequence not specifically defined in the toolbox. See Chapter 5 for a list of the keys defined to ESP.

When GTSCR returns to the application, the field number the cursor is in and the character causing the return are passed back to the application. This is how the application can use function keys and control characters to control program execution.

Return Caused by Undefined Character

If the return was caused by an undefined key being pressed, that character's ASCII value is in the low order byte of the char variable, as discussed above. You can test this value with this statement:

```
IF (CHAR AND 255) = XXX THEN ...........
```

xxx represents the value of the character you want to test for.

[ESCAPE] followed by a control character may be the same as a function key. See Chapter 5. The ASCII values for these characters is 128 plus the value of the control character.

For [ESCAPE] followed by any other character, the value passed back to the program is 128 plus the ASCII value of the other character. For instance, [ESCAPE] [A] returns 193.

For example, the following checks to see if the return to the application was caused by the [MENU] key:

```
IF (CHAR AND 255) = ESP'MENU THEN ..........
```

As discussed in Chapter 5, ESP'MENU is defined in the SYSSTD.BSI map statement module included with ESP and has a value of 155. this is the function key's ASCII value—equivalent to [ESCAPE] plus [ESCAPE], which translates to 128 plus 27.

Return Caused by Screen-generated Interrupt

If return to the application was caused by a screen generated interrupt (due to a field having the pre-edit, post-edit, or changed value post-edit interrupt field flag set), then the bit value of the interrupt is in the high order bits of the character variable. The value is either:

- 32768, defined in SYSSTD.BSI as ESP'POSTEDIT.
- 16384, defined in SYSSTD.BSI as ESP'PREEDIT.

Testing for these characters is discussed in "Special Processing Flags," above.

SCREEN OUTPUT

To make ESP as efficient as possible, GTSCR keeps track of all screen output so unnecessary screen redisplay is eliminated.

When the application program loads a field with SETVAL, the field is flagged for redisplay and is redisplayed the next time GTSCR is called.

The problem with this technique is that if the application modifies the screen directly to change data displayed by GTSCR, the display is not an accurate representation of the data in the screen.

In order to deal with this situation, you can explicitly tell GTSCR to redisplay all fields by calling GTSCR and specifying only the screen argument. This re-coordinates the screen's image with the data in the screen. You can also use the SETDSP XCALL to set all fields to redisplay, then call GTSCR.

We strongly suggest that the application program does no screen output except through ESP because screen prints and updates will not properly reflect the information on the screen.

REDISPLAYING A SCREEN

The preferred method to redisplay the screen is:

```
XCALL GTSCR, screen
```

This has the same effect as setting char to ESP'REPAINT and calling GTSCR in the normal way.

The preferred method to update the display for changed fields is:

```
X = ESP'MENU
XCALL GTSCR, screen,X,1
```

HIDFLD - HIDE SCREEN FIELDS

The HIDFLD XCALL lets you hide or show one or more screen fields. When you hide a field, it does not display and cannot be edited. When you show a field, it displays (unless it is read secured) and can be edited if it is an editable field.

You can conditionally hide and show fields using field expressions. See the *SEDIT User's Guide* for more information.

To call HIDFLD, use this format:

```
XCALL HIDFLD, screen, flag, field {,field,...}
```

screen is the variable holding the screen file.

flag determines if the field(s) are hidden or shown. If flag is true (non-zero), the fields are hidden; if flag is false (zero), the fields are shown.

field,field,... is a list of field numbers. You must include at least one field number. All fields are hidden or shown based on the flag value.

INITRM - INITIALIZE THE TERMINAL

You use the INITRM XCALL to prepare the terminal to use the special functions associated with ESP.

INITRM performs the following functions:

- Puts the terminal into single character input mode.
- Disables character echoing.
- Sets up the function key translation table.
- Clears the screen.
- Sets up the top and bottom status lines.
- Clears accumulators @0 through @249.
- Sets the default foreground and background colors.
- Selects seven-bit or eight-bit characters.
- Reads printers definitions from the PRTDEF.lan file.

To call the INITRM subroutine, use this syntax:

```
XCALL INITRM {, top'msg${, bottom'msg$}} {flags}
```

top'msg$ is a string variable or expression to display on the top status line. If the terminal does not have a top status line, this argument is ignored.

top'msg$ is most effective as a short description of the current function being performed by the application. This makes the application consistent and informative to people using it.

For no message display, specify a null string.

bottom'msg$ is a string variable or expression to display on the bottom status line. You can specify any text you feel is appropriate to your application. For example, you may want to display the user's company name.

For no message display, specify a null string.

The exact format for the status line display is discussed below.

flags let you choose which of the INITRM functions listed above, such as reading printer definitions, are actually performed. The flags are:

INITRM'NCA	Do not clear accumulators.
INITRM'NCS	Do not clear screen.
INITRM'NPD	Do not read printer definitions.
INITRM'NSL	Do not change status lines
INITRM'RST	Reset terminal status; exit data mode and allow standard input.

STATUS LINES

INITRM sets up the top and bottom status lines for terminals supporting them. Optionally, you can display messages on either or both status lines.

If you supply neither top nor bottom status line messages, INITRM clears both status lines completely. If you supply only the top status line message, INITRM clears the bottom status line completely.

You must have a top status line message text argument if you want to display a bottom status line message. The top status message text argument may, however, be a null string.

If you specify the top status line message text argument, INITRM displays your application name at the left of the top status line. At the right it displays the device and account where the user is logged. The text argument is displayed to the left of the application name separated by a dash, and can be up to 50 characters long.

If you include a bottom status line message text argument, INITRM displays the current date and time at the right of the bottom status line. The maximum number of characters for the text argument for the bottom status line is 62 characters.

INPREC - READ RECORD FROM SEQUENTIAL FILE

The INPREC XCALL reads a specified number of bytes from an open sequential file. The calling format for INPREC is:

```
XCALL INPREC,channel#,ret'variable,record'size
```

channel# is the channel number of the open sequential file.

ret'variable is a string or unformatted variable to hold the bytes read from the file. It must be at least as large as record'size. If it is larger than record'size, it is padded with nulls.

record'size is the number of bytes to read.

IREAD - READ AN ISAM DATA RECORD

The IREAD XCALL reads records from previously opened ISAM files. It is intended to replace the READx AlphaBASIC statements.

IREAD does not work with ISAM PLUS files.

IREAD reads an ISAM data record from the disk into any string or X formatted variable which is at least as long as the record. Optionally, it leaves just the ISAM data file physical record locked by LOKSER.

Just as READx statements require an ISAM 1, 2, or 7 to precede them, so IREAD requires a record be found by ISAM before the read is attempted. In all cases, you may use IREAD to replace READx statements.

To call the IREAD XCALL from your AlphaBASIC program, use this syntax:

```
XCALL IREAD, channel, record, error, lock'var
```

channel is a numeric value or expression containing the channel number of the ISAM file being accessed.

record is an X format variable in which the disk record is returned. It must be at least as long as the record being read.

error is a variable loaded with the ISAM error code value if the disk read is unsuccessful. It is zero if the read succeeds. See the *ISAM System User's Guide* for error code definitions.

lock'var is a variable or expression passed to the subroutine, whose value is an integer from -2 through 1:

- -2 relocks the index without reading the record.
- -1 leaves the data file record locked and unlocks the index file.
- 0 indicates no record locking is to take place.
- 1 leaves both the index and data records locked.

ISMROK - READ THE ISAM ROCK VALUES

The ISMROK XCALL subroutine reads the ISAM rock and loads the rock values into the specified variables.

A sample program named ISMROK.BAS is included with the ESP software to clearly illustrate using ISMROK.

ISMROK does not work with ISAM PLUS.

To call the ISMROK subroutine, use this syntax:

```
XCALL ISMROK, channel, var{, var ...}
```

channel is a numeric variable or expression specifying the channel number on which the ISAM file is open.

var is one or more of the following variable names, to be loaded with the indicated information. Except where noted, they are all floating point variables:

IDAALC	Number of allocated data records.
IDAAVL	Number of available data records.
RECSIZ	Byte size of the data record.
KEYSIZ	Byte size of the ISAM key.
KEYPOS	Position of the key in the data record.
DEVDRV	Device name and drive number where the data file resides. This is a ten byte string variable.
IDXALC	Number of allocated index blocks.
IDXAVL	Number of unused index blocks.

JULCVT - CONVERT DATE TO/FROM JULIAN

The JULCVT XCALL converts dates from internal AlphaBASIC (separated) format to Julian dates, and vice-versa. A Julian date is the date expressed as the number of days since January 1, 4513 BC. Expressing dates in Julian format allows you to perform mathematical comparisons and operations more easily.

The calling format for JULCVT is:

```
XCALL JULCVT,date,julvar{,opcode{,status}}
```

The ***date*** variable contains the date in separated format.

julvar contains the Julian date.

opcode is an optional variable defining the direction of the conversion. 1 indicates conversion form Julian to separated format. If opcode is zero, or omitted, the date is converted from separated to Julian.

status is an optional variable to hold a completion status code. If status is zero, conversion was successful. if status is a positive number, an error occurred; the value of status is the error number.

JUSTXT - JUSTIFY A STRING

The JUSTXT XCALL lets you justify the contents of a string or unformatted variable in one of four ways: right, left, centered, or repeated.

The calling format for JUSTXT is:

```
XCALL JUSTXT,string,result,type
```

string is a string or unformatted variable containing the text string to justify.

result is the string or unformatted variable used to return the justified result.

The variable used to return the justified result must be at least as large as the variable with the string to be justified.

The ***type*** variable tells JUSTXT which type of justification you want. These flags are defined in SYSSTD.BSI to indicate the types:

JT'CEN	Centers the text.
JT'LFT	Left justifies the text.
JT'RGT	Right justifies the text.
JT'RPT	Repeats the text as many times as will fit in the result variable.

LSTBOX - PROCESS A LIST BOX

The LSTBOX XCALL subroutine provides a way to let a user select from a list of items that may be too large to display on one screen or in a small portion of a screen. One particular use would be for a list of items that changes from user to user or from session to session where it is impractical to use pop-up items or the on-line help message to list the choices.

LSTBOX is extremely flexible. Since you design the list box using SEDIT, you decide how many items are shown at a time, the length of the fields, where on the screen the list appears, and so on. How to create a screen to use as the list box is described below. LSTBOX also lets you set the step rate—how many lines the box scrolls when the user reaches the end of the displayed selections.

You can use LSTBOX in two ways:

- By itself. In this case, you fetch the list box screen like any other, then use LSTBOX to open and display it.
- More usually, you'll want to open a list box for a particular field of another screen that is already open. In this case, the field you open the list box for must have the pre-edit interrupt flag set, as described in the *SEDIT User's Guide*, so GTSCR returns to the calling program when the cursor enters the field, and your program can call LSTBOX to process the list box screen.

Since using LSTBOX involves several steps, we'll describe the format of the XCALL, then, after talking about the list box screen, list the steps you must follow in your program to use LSTBOX properly.

Calling the LSTBOX routine does not automatically display the list box screen; LSTBOX is the routine to use for a field where the user *may* want to display a list box. When you use LSTBOX to process a field, the user can press a key to display the list box. See "Using a List Box Screen," below.

The calling format for LSTBOX is:

```
XCALL LSTBOX,lstbox'screen,item'list,LSTBOX'PARM,char,{usrscr,usrfld}
```

lstbox'screen is an unformatted variable containing the list box screen. The screen should not be opened before calling LSTBOX. LSTBOX opens and closes the list box screen as it's needed.

item'list is an unformatted variable containing the list of selections to scroll in the list box. Each selection must be a null-terminated text string. The last selection must be followed by two nulls.

LSTBOX'PARM is the unformatted variable defined in the LSTBOX ++INCLUDE file, described below.

char is a floating point variable to hold the character which causes the return from LSTBOX.

When calling LSTBOX while processing another screen, ***usrscr*** is the unformatted variable which contains the main screen being processed. This should be the same variable you used in the FETCH call for the main screen.

usrfld contains the field number of the field for LSTBOX to process. As mentioned above, this field must have the pre-edit interrupt flag set.

When LSTBOX returns:

char contains the character that caused LSTBOX to return to your application. This is the character the user entered to leave the field or [MENU] if the user aborted either the list box or the main screen.

The number of the selection the user chose is in the LB'SEL subvariable of LSTBOX'PARM. If the user pressed [MENU] to exit the list box without making a selection, LB'SEL is zero.

DESIGNING A SCREEN TO USE WITH LSTBOX

The list box displayed by LSTBOX is a standard ESP screen file created by SEDIT. This lets you design the list box to meet your needs. You can set the number of choices shown at once, whether to use one or two columns, and so on. When designing the screen, you must follow these rules:

- The fields used for the selection items must be contiguous and must be non-changeable, so the user can move the cursor to them but cannot modify them.
- The selection fields can be any length but the shorter they are the faster LSTBOX can scroll the contents of the list box.
- All other fields must be display only.

IMPLEMENTING LSTBOX IN YOUR PROGRAM

There are several steps to processing a list box using LSTBOX:

1. You must ++INCLUDE the file LSTBOX.BSI, or .BPI for AlphaBASIC PLUS, into your program. This file defines the parameter structure required for LSTBOX. The MAP1 variable name from this file is the LSTBOX'PARM variable passed in the XCALL.
2. Load the list box screen into an unformatted variable using XCALL FETCH. Do not open the screen.
3. Load the list of selections for the list box into an unformatted variable. As described above, each element in the variable must be null terminated with the last one terminated by two nulls. Since LSTBOX returns a number to indicate which choice was made, your application needs to keep track of the order of the selection items.

4. Set the step rate using the LB'STP variable in the LSTBOX'PARM parameter structure. This step rate defines how many lines the display scrolls when the user attempts to move beyond the top or bottom of the list box screen. If you specify one, the display will only scroll one line at a time. If you specify more than one, the display will scroll that many lines. However, regardless of the number of lines scrolled, the cursor always goes to the next selection in sequence.

   You cannot specify a step rate greater than the number of selection fields in the list box screen.

5. Set the flag values you want into the LB'FLG variable of LSTBOX'PARM. The flags are:

LF'RTN	Return to calling program when the user closes the list box. See "Using a List Box Screen," below.
LF'LOD	Display the selection in the screen field when the user closes the list box.

   If you want the user to press something other than [↑] and [↓] to display the list box, set the LB'CHR unformatted variable to the character you want.

6. Call LSTBOX.

7. When LSTBOX returns to your program, the number of the user's selection is in the LB'SEL subvariable of LSTBOX'PARM. If you set the LF'LOD flag before calling LSTBOX, the selection is also displayed in the field of the main screen.

USING A LIST BOX SCREEN

To display a list box, the user should press [↑] or [↓] in the list box field (unless you've defined a different access character, as described above).

Pressing [↑] or [↓] moves the cursor through the selections; when the cursor reaches the end of the screen, the selections will scroll. To select an item, move the cursor to it and press [EXECUTE] or [RETURN]. To exit the list box without making a selection, press [MENU].

Whether the cursor stays in the list box field or moves on when the user closes the list box is determined by the LF'RTN flag of the LB'FLG variable in LSTBOX'PARM, discussed above. If this flag is set, closing the list box causes LSTBOX to return to the calling program, and you can take appropriate action depending on the character causing the return. If LF'RTN is not set, closing the list box does not exit the LSTBOX subroutine; the cursor stays in the list box field of the main screen, and the user needs to press a key ([MENU], [EXECUTE], [↓], etc.) to move out of the field and cause a return to the calling program.

OPNSCR - OPEN A SCREEN FOR PROCESSING

The OPNSCR XCALL opens an ESP screen for display. ESP maintains a list of currently open screens, to ensure that windowing and overlaying of screens is handled properly. OPNSCR places a screen in this list.

OPNSCR does not actually display the screen; use GETSCR to do so.

To open a screen, use this syntax:

```
XCALL OPNSCR, screen
```

screen is the name of an X format variable containing the screen file.

OPNTMP - GET TEMPORARY FILE EXTENSION

The OPNTMP XCALL subroutine returns a file name with the first available extension in the series .000 through .999. It is primarily used for print files. This method of naming print files is used by AlphaWRITE and AlphaCALC when printing a document or worksheet. The file that is created on the disk and subsequently spooled has an extension of .000, .001, .002, etc.

You pass OPNTMP a file name without an extension; OPNTMP checks the directory of the current account and returns the first available numeric extension for that file name.

The calling format for OPNTMP is:

```
XCALL OPNTMP,filename
```

filename is a string variable containing a file name with no extension. OPNTMP returns the file name with the selected extension in this variable, so it must be at least ten characters long.

PRTSCR - PRINT AN IMAGE OF THE SCREEN

The PRTSCR XCALL subroutine prints an image of the screen, or of only selected field classes of the screen. PRTSCR either puts this information in a file opened by the application or spools the screen image directly.

PRTSCR creates a screen image and either creates a file to be spooled directly, or places the screen image in the file referenced by the channel number provided in the XCALL. PRTSCR does not print trailing blank lines.

To call PRTSCR, use this syntax:

```
XCALL PRTSCR,screen{,channel{,linctr{,lincnt{, chrcls}}}}
```

screen is an X format mapped variable containing a screen created with ESP, previously loaded using ESP's FETCH AlphaBASIC XCALL subroutine.

channel is a numeric variable or expression specifying the channel number of the previously opened sequential file to which the screen will be printed.

linctr is a floating point variable incremented by the number of lines printed.

lincnt is a floating point variable to be loaded with the number of lines printed.

chrcls is a floating point variable used to describe two different parameters: the fill character and the field class(es) to be printed.

The fill character specifies the character used to fill in unused space within a field. It is typically a space (decimal 32). This parameter is contained in the low order byte of the chrcls value.

The field class to be printed is in the high order byte of the chrcls value. Valid class definitions in SYSSTD.BSI are:

ESP'FLRCHR	Filler character (space)
ESP'PRTTTL	Print titles
ESP'PRTHDR	Print detail headers
ESP'PRTDET	Print detail level
ESP'PRTST1	Print subtotal level 1
ESP'PRTST2	Print subtotal level 2
ESP'PRTST3	Print subtotal level 3
ESP'PRTGTL	Print grand totals

You can print several field classes at one time by summing the appropriate code values together and passing that sum as an argument to PRTSCR.SBR. If no class specifier is given (the high order byte of chrcls is cleared), all classes are printed.

If chrcls is omitted, PRTSCR defaults to a fill character of SPACE (decimal 32) and prints all field classes.

EXAMPLES

To spool a complete image of the screen directly:

```
XCALL PRTSCR, SCREEN
```

To print title and detail fields together, such as you might want to do at the start of a report, use the following sample code as a guide:

```
MAP1 ESP'PRTTTL     , F, 6, (1*256)     ! Title class
MAP1 ESP'PRTDET     , F, 6, (4*256)     ! Detail class
MAP1 ESP'FLRCHR     , F, 6, ASC("")     ! Filler character

CHRCLS = ESP'FLRCHR + ESP'PRTTTL + ESP'PRTDET
XCALL PRTSCR, SCREEN, CHANNEL, LINCTR, LINCNT, CHRCLS
```

The MAP1 statements above, along with others defining other field classes for reports, are included in the SYSSTD.BSI file.

To print detail fields only:

```
CHRCLS = ESP'FLRCHR + ESP'PRTDET
XCALL PRTSCR, SCREEN, CHANNEL, LINCTR, LINCNT, CHRCLS
```

To print a detail field padded with asterisks:

```
CHRCLS = ASC("*") + ESP'PRTDET
XCALL PRTSCR, SCREEN, CHANNEL, LINCTR, LINCNT, CHRCLS
```

To detect page breaks, use the linctr and lincnt variables. lincnt contains the number of lines output to the file open on channel during the last call to PRTSCR. linctr contains the accumulated number of lines output to the file.

By comparing linctr + lincnt to the number of lines allowed on a page, you can determine if there are sufficient lines remaining to support another call to PRTSCR. For example:

```
MAIN'REPORT'LOOP:
      .
      .
      XCALL SETVAL, PRTSCR, FLD'NBR, FLD'001, FLD'002......
      CHRCLS = ESP'FLRCHR+ESP'PRTDET        ! Set for detail.
      IF (LINCTR + LINCNT) > LIN'PER'PAG THEN CALL PAGE'BREAK
      XCALL PRTSCR, SCREEN, CHANNEL, LINCTR, LINCNT, CHRCLS
      GOTO MAIN'REPORT'LOOP
      .
      .
PAGE'BREAK:
      PRINT #CHANNEL, CHR$(12);        ! Issue form feed
      LINCNT = 0                       ! Clear line count.
      CHRCLS = CHRCLS + ESP'PRTTTL     ! Flag for title print.
      RETURN                           ! Return to main loop.
```

For more complete examples of how to use the PRTSCR subroutine, please examine the file CSTPRT.BAS, distributed with the Customer Maintenance Package.

PRTSEL - SELECT A PRINTER

The PRTSEL XCALL displays a printer selection screen and lets the user select these printing options:

- The destination printer.
- Whether to spool the output file.
- The number of copies to print
- What form name to use.

The screen PRTSEL displays is in the screen file DMSESP:PRTSEL.lng, where *lng* is the extension for the current language. PRTSEL fetches and opens this file itself; you do not have to do so before calling PRTSEL.

The list of printers displayed is taken from the user impure area of TOOLBX.SYS. Printer names are loaded into this area by INITRM.

The calling format for PRTSEL is:

```
XCALL PRTSEL,prt'name,spl'flag,copies{,form}
```

prt'name is a string variable of at least six characters. When you call PRTSEL, this should contain the name of the default printer for the application. When PRTSEL returns to the application, prt'name contains the name of the selected printer.

spl'flag is a numeric variable which returns a value indicating if the file is to be spooled:

- 0 indicates the file should be spooled.
- -1 means do not spool the file.
- -2 indicates the user pressed MENU to abort PRTSEL.
- A positive number indicates an error. The value is the error number.

copies returns the number of copies to print.

form is an optional string variable which returns the form name selected.

RREAD - READ RANDOM FILE RECORD

The RREAD XCALL reads a record from a random file into a string or unformatted variable. You can lock the record after the read operation. The calling format for RREAD is:

```
XCALL RREAD,channel#,record'buffer,return'code{,lock'flag}
```

channel# is the channel number of the file, which must already be open.

record'buffer is the variable to read the data into. It must be at least as long as the record being read.

return'code returns the error status of the read operation.

The optional ***lock'flag*** variable indicates whether you want to lock the record. If it is 0, the record is not locked; if it is anything else, the record is locked.

SELFLG - PROCESS SELECTION SCREEN

The SELFLG XCALL lets you create a screen where the user can highlight one or more choices from a list. For example, the field flag, field class, and field modifiers fields of the SEDIT Field Editor screen use SELFLG-type screens.

You must FETCH the screen file for SELFLG before calling this routine. Do not use OPNSCR; SELFLG handles opening and closing the screen.

The calling format for SELFLG is:

```
XCALL SELFLG, screen, fld'flag'table(1,1), flags
```

screen is an X format variable containing the screen file for the SELFLG screen.

fld'flag'table is a two-dimensional array containing the selection field numbers and flag values. It is discussed in detail below.

flags is a variable of the same type as fld'flag'table, containing the current flag settings. When SELFLG returns to the calling program, flags contains the new flag settings.

The next section describes creating and using a selection screen file; after that, we describe defining information about that screen to your program in the fld'flag'table variable.

THE SELFLG SCREEN FILE

A screen intended for use with SELFLG should have only two types of fields on it: display-only fields, such as the screen title, boxes or lines, and so on; and the selection fields themselves.

The selection fields must be non-changeable; the user must be able to move the cursor into them, but not allowed to make an entry. They must also be consecutively numbered.

There should not be any entry fields on a selection screen; if the selections are part of a larger entry process, make the selection screen a separate file which is windowed over the entry screen.

To make selections on this type of screen, the user can use [TAB], [↑], or [↓] to move between fields, and press [SPACE] to select or deselect the current field. Pressing [EXECUTE] leaves the screen and saves the new settings; pressing [MENU] leaves the screen without keeping any changes.

THE FIELD FLAG TABLE

As mentioned above, the fld'flag'table variable is a two-dimensional array. It can be either a binary or integer variable type, defined by one of these two MAP statements:

```
MAP1 FLD'FLAG'TABLE(n,2),B,4

MAP1 FLD'FLAG'TABLE(n,2),I,4
```

n is the number of selection fields on the screen.

The two elements of each item in the array are the field number of the field and a flag value for that field number. So, loading the information into the array would look like this:

```
FOR I = 1 to n
     FLD'FLAG'TABLE(I,1) = field #
     FLD'FLAG'TABLE(I,2) = flag value
NEXT I
```

Because you must load field numbers into this variable; more information about your screen is "hard-coded" into your program than with other ESP screens. Whenever you add or change a field number, this section of the code needs to be modified.

SETDFC - SET SPECIAL DISPLAY CHARACTERISTICS

The SETDFC XCALL lets you change the display format code and the foreground and background colors for a field in a screen created using ESP.

For example, you can use SETDFC to change a field's display attributes from normal to reverse video or to underline the field.

You can use SETDFC any time after the screen is initialized with the INITRM XCALL. The changed display format is not reflected on the screen until you call GTSCR to do screen processing.

To call the SETDFC subroutine, use this syntax:

```
XCALL SETDFC, screen field'number, start'code, end'code,
back'color, fore'color
```

screen is an X format mapped variable containing an ESP screen previously loaded using the FETCH XCALL.

field'number is a floating point variable or expression specifying the number of the field to change.

start'code is a variable or expression whose value is the new starting display code for the field specified by field'number. Allowable values are the same values you can use at the SEDIT Field Editor's start format code field. The display codes are:

DISPLAY FEATURE	START CODE	END CODE
Blink	21	22
Underscore	30	31
Reverse Video	32	33
Reverse Video Blink	34	35

end'code is a variable or expression whose value is the new ending display code for the field specified by the field'number. These are the same values as used at the SEDIT Field Editor's end format code field. The codes are listed above.

back'color is a variable or expression whose value is the new background color for the field specified by field'number. These are the same values you use at ESP's Field Editor background color field. These are the color codes:

VALUE	COLOR
0	Black
1	White
2	Blue
3	Magenta
4	Red
5	Yellow
6	Green
7	Cyan

Values 8 through 15 are dim versions of the same colors.

Many terminals let you select display colors from a palette of available colors. If the terminal colors have been changed from the default, these values may select colors other than those shown in this table.

fore'color is a variable or expression whose value is the new foreground color for the field specified by field'number. These are the same values as used at ESP's field editor foreground color field when the screen was created. See the table above for values.

SETDFP - CHANGE A FIELD'S SCREEN POSITION

The SETDFP XCALL lets you change the position of a field on an ESP screen. Its calling format is:

```
XCALL SETDFP,screen,field,row,column
```

screen is an unformatted variable containing the ESP screen. The screen can be opened, but it doesn't have to be.

field is a floating point variable containing the field number of the field you want to change the position of.

row is the new row position to be set.

column is the new column position to be set.

SETDSP - SET ALL FIELDS FOR REDISPLAY

As described in the GTSCR section, GTSCR normally redisplays only those fields on a screen which it knows have changed. The SETDSP XCALL sets all fields on an ESP screen to be redisplayed when you call GTSCR.

The calling format for SETDSP is:

```
XCALL SETDSP,screen
```

screen is an unformatted variable containing the ESP screen.

SETFSC - SET FIELD SECURITY

The SETFSC XCALL lets you set read and write security for a specific field of an ESP screen.

You can permanently set field read and write security using SEDIT's Field Editor screen. See the *SEDIT User's Guide*.

SETFSC's calling format is:

```
XCALL SETFSC,screen,field,read'sec,write'sec
```

screen is an unformatted variable containing the screen file.

field is the number of the field to set the security for.

read'sec is the read security value to set. It can be from 0 through 255.

write'sec is the write security value to set. It can be from 0 through 255.

SETPUI - SET POP-UP ITEM

The SETPUI XCALL lets you set a value into a pop-up item field. Its format is:

```
XCALL SETPUI,screen,field,value
```

screen is the unformatted variable holding the screen file.

field is the number of the field you want to set the pop-up item value for. If a variable, it can be either floating point or integer format.

value is the value you want to put in the field. If it is a variable, it should be a floating point variable. The value should be a positive integer from one through the number of pop-up items possible in the field.

SETPUI changes the field's value, but does not redisplay it; you must call GTSCR to redisplay the screen. This will display the pop-up item corresponding to the number you set using SETPUI.

SETUSR - SET USER IMPURE ELEMENTS

The SETUSR XCALL subroutine writes a specified value to the user impure area in TOOLBX.SYS. The user impure area is set up when TOOLBX.SYS is loaded into system memory during booting. You can read elements of the user impure area using the GETUSR XCALL, described earlier in this chapter.

To call the SETUSR subroutine, use this syntax:

```
XCALL SETUSR, element'number, var
```

element'number is a floating point variable or expression specifying which element of the user impure area to access.

The active element numbers are 1 through 22, 1000 through 1009, and 10000 through 10511. They are described below.

var is a variable or expression whose value is loaded into element'number.

For element numbers 1, 2, 3, 5, 6, 8, 9, 10, and 11, var must be a floating point variable.

For element number 4, var must be a string of at least 6 bytes.

For element number 7, var must be a string of at least 10 bytes.

For elements 12, 13, and 14, var must be a string variable.

SETUSR Elements

The following list describes all the active SETUSR element numbers:

- Element 1 is a decimal number which sets the following flags:

VALUE	INDICATES
1	User is on a modem; do not display on-line help messages.
2	Do not spool ESP screens; print to disk file only.
4	Use eight-bit characters.
32768	Allow CTRL / C during SIGNIN.

The value for element 1 should be the sum of all the flags you want set.

Always read element 1 before loading it and make sure you change only the flag(s) you want to.

- Element 2 is the user's read security level. It can be from 0 through 254.
- Element 3 is the user's write security level. It can be from 0 through 254.

Remember: the higher the value, the greater the user's security clearance.

- Element 4 is the user's default printer name. It can be any valid printer name.

- Element 5 is the application timeout period in seconds, from 0 to 65535. A value of 0 means the application never times out.

- Element 6 is the ASCII character to send when the application times out. It can be from 0 to 255. The most likely value is 155, which represents the [MENU] key. However, any other character is valid and may be used. This element may be left blank. Chapter 5 discusses ESP's handling of special keys.

- Element 7 is the user's name, which may be any combination of ASCII characters up to 10 characters long. This element may be blank.

- Elements 8 and 9 are for color control on terminals supporting color. They may be from 0 to 255. Element 8 sets the default background color; element 9 sets the default foreground color. The color selections for foreground and background follow Alpha Micro's standards.

  The standard color definitions and values are (these may be changed on some color terminals):

VALUE	COLOR
0	Black
1	White
2	Blue
3	Magenta
4	Red
5	Yellow
6	Green
7	Cyan

  Colors 8 through 15 are the dim versions of these colors.

- Element 10 is the current company number. This value is normally maintained by SIGNIN, if active.

- Element 11 is the last menu selection made by the user from the SIGNIN menu. This value is normally maintained by SIGNIN, if active.

- Element 12 is the current company name, which may be any combination of ASCII characters up to 60 characters in length. This value is normally maintained by SIGNIN, if active.

- Element 13 is the current default help file specification.

- Element 14 selects and begins indirect input from the specified sequential file. The default extension is .TIP (**T**erminal **I**n**P**ut). Until the end of the file is reached, any ESP application will take input from the file instead of from the keyboard.

  To specify function keys in the indirect input file, use the format [ESCAPE] [x], where ***x*** is the function key number expressed as an ASCII value. For example, [F1] is ASCII 27 followed by ASCII 1.

  Element 17 lets you set indirect input from a string rather than from a file.

- Element 15 sets the user's level, from 0 through 100.

- Element 16 sets the user's expertise, from 0 through 100.

- Element 17 sets indirect input from a string variable. You may set this element multiple times; each string is linked to the next in a last in, first out chain.

- Element 18 clears an indirect input level.

- Element 19 sets the number of iterations for the current indirect input string (the last one added using element 17).

- Element 20 sets the keystroke record file. This file is placed first in the current indirect input list.

- Element 21 clears a keystroke record level.

- Element 22 sets the current report page number. This is useful only with the DART data base manager.

- Element 1000 lets you set or clear a JLOCK.

This is supported for compatibility with previous versions. We strongly recommend you do not use this element for new application development.

- Element 1001 loads the SSD number into the user impure area.

- Element 1002 changes the current device and account number.

- Element 1003 sets up the JCB according to MUSER data. var should be the user name.

- Element 1004 puts a message in the OPR:MSGLOG.FIL file.

- Element 1005 requests the OPR:MSGLOG.FIL file.

- Element 1006 releases the OPR:MSGLOG.FIL file.

Since element 1004 locks OPR:MSGLOG.FIL before writing to it, and releases it when it is through, you need to use elements 1005 and 1006 only if you want to perform on operation on the file outside of ESP control.

- Element 1007 clears the OPR:MSGLOG.FIL file.
- Element 1008 sets the user's debug level.
- Element 1009 sets a flag indicating whether SIGNIN is active; if it is, set the value to 1, if not, set it to 0.
- Elements 10000 through 10511 are floating point values corresponding to ESP expression accumulators @0 through @511. See the *SEDIT User's Guide* for information on accumulators.

SETVAL - SET VALUES INTO SCREEN FIELDS

The SETVAL XCALL subroutine loads values into fields in a screen created using ESP. The screen must have been previously loaded into a variable using the FETCH XCALL. You can then use SETVAL to set values into specific fields of the screen contained in the variable.

SETVAL loads a specified set of AlphaBASIC variables into fields within the ESP screen specified in the calling sequence. It then converts, as necessary, the AlphaBASIC variables into the appropriate field format(s).

SETVAL only sets the specified fields to the specified values in the screen variable; it does not actually update the screen display. All display manipulation is handled by the GTSCR XCALL subroutine, described earlier in this chapter.

If a date field has the sorting format return field modifier flag set, then any data sent to the field must be in YYMMDD format.

To call the SETVAL subroutine, use this syntax:

```
XCALL SETVAL, screen, field'number, var1{, var2,..., varn}
```

screen is an X format variable containing a screen created using ESP, previously loaded using ESP's FETCH XCALL.

field'number is a floating point variable or expression specifying the field number at which to start loading values from the ESP screen specified in screen.

var1 is a variable or expression whose value is loaded into field'number. It can be any AlphaBASIC variable type.

var2 is an optional variable or expression whose value is loaded into the next sequential field after field'number.

You may specify any number of additional variables. Each additional variable is loaded into the next sequential field.

For example, if field'number is 5, var1 is loaded into field number 5, var2 is loaded into field number 6, and so on. This syntax is provided for convenience and may, but does not need to, replace multiple XCALL SETVAL statements.

SETYNF - SET A YES/NO RESPONSE INTO A FIELD

The SETYNF XCALL allows your program to set the value of a yes/no type field. A yes/no field is defined by setting the yes/no option in the field modifiers for a text field. The only characters a user can enter in this type of field are the first letters of yes or no as defined for the user's current language.

The format for calling SETYNF is:

```
XCALL SETYNF, screen,field'number,setting
```

screen is an X format variable containing an ESP screen, previously loaded using ESP's FETCH XCALL.

field'number is a floating point variable or expression specifying the field number of the yes/no field.

setting is a floating point variable containing the setting. Set this to 1 for yes or 0 for no.

SKPFLD - SKIP SCREEN FIELDS

The SKPFLD XCALL lets you skip or allow one or more screen fields. When you skip a field, it still displays but the cursor will not go to it for editing. When you allow a field, it can be edited unless it is write secured.

You can conditionally skip and allow fields using field expressions. See the *SEDIT User's Guide* for more information.

To call SKPFLD, use this format:

```
XCALL SKPFLD, screen, flag, field {,field,...}
```

screen is an X format variable holding the screen file.

flag determines if the field(s) are skipped or allowed. If flag is true (non-zero), the fields are skipped; if flag is false (zero), the fields are allowed.

field,field,... is a list of field numbers. You must include at least one field number. All listed fields are skipped or allowed based on the flag value.

Both flag and field can be either floating point or integer variables.

TOOLBX - MISCELLANEOUS TOOLBX FUNCTIONS

The TOOLBX XCALL is slightly different than most other ESP XCALLs. Instead of performing a specific function, it lets you perform different operations depending on the opcode you use. There are currently three supported opcodes:

- TBX'FLDCNT to find the number of fields on an ESP screen.
- TBX'GETOLH to retrieve the on-line help message for a field.
- TBX'SQFLSZ to get the size of a sequential file, in bytes.

The following sections describe how to use each of these opcodes.

TBX'FLDCNT: NUMBER OF SCREEN FIELDS

The format for the TOOLBX XCALL with the TBX'FLDCNT opcode is:

```
XCALL TOOLBX TBX'FLDCNT,screen,count
```

screen is an unformatted variable holding the screen file you want the number of fields for.

count is a floating point variable into which the number of fields is returned.

TBX'GETOLH: GET ON-LINE HELP MESSAGE

The format for the TOOLBX XCALL with TBX'GETOLH is:

```
XCALL TOOLBX TBX'GETOLH,screen,field,text
```

screen is an unformatted variable holding the screen file.

field is the number of the field you want the help text for. If it is a variable, it must be floating point.

text is a string variable into which the on-line help message for the indicated field is returned. The message is null-terminated.

TBX'SQFLSZ: SEQUENTIAL FILE SIZING

The format for the TOOLBX XCALL with TBX'SQFLSZ is:

```
XCALL TOOLBX TBX'SQFLSZ,file,size
```

file is a string variable or literal containing the file specification of the file you want the size for.

size is a floating point variable into which the size of the file, in bytes, is returned.

CHAPTER FIVE
SPECIAL KEY HANDLING

When someone is using an ESP screen, all input handling is taken care of by the TBX$GETSCR subroutine (for assembly language applications) or the GTSCR XCALL (for AlphaBASIC applications). However, certain keys cause ESP to return to the application so the application program can take appropriate action. This chapter discusses what those keys are and what you may need to do in your software to interpret them properly.

WHEN YOU RETURN TO YOUR APPLICATION

ESP (specifically, GTSCR or TBX$GETSCR) returns to your application whenever a user presses a control character or escape sequence other than the ones listed in the table below.

CHARACTERS INTERPRETED BY ESP

The control characters and escape sequences in this table are defined by ESP. They are interpreted by GTSCR (or TBX$GETSCR), and are not returned to the application program. The key names in the center column are for the AM-62, AM-65, AM-72 and AM-75 family of terminals, and may not be accurate if your application uses its own function key translation file, as described in the next section.

FUNCTION	KEY NAME	CONTROL SEQUENCE
Clear to end of line	None	CTRL / Y
Delete character	CHAR DEL	CTRL / D
Insert character	CHAR INS	CTRL / F
Delete word	WORD DEL	CTRL / V
Move cursor left	←	CTRL / H
Move cursor down	↓	CTRL / J
Move cursor up	↑	CTRL / K
Move cursor right	→	CTRL / L
Next word	NEXT WORD	CTRL / W
Previous word	PREV WORD	CTRL / A
Character insert mode	None	CTRL / Q
Beginning of field	SHIFT / ←	CTRL / U
End of field	SHIFT / →	CTRL / N
Next field	TAB	CTRL / I
Previous field	BACK TAB	CTRL /]
End of entry	RETURN	CTRL / M
Help	HELP	ESCAPE ?
Desk accessories	SHIFT / F13	ESCAPE @
80/132 column mode	SHIFT / F16	ESCAPE \

Any control character or escape sequence other than the ones in this list are passed through to the application.

Also, remember that any key which takes the cursor out of a field, such as TAB, may cause a return to the application program if the post-edit interrupt flag is set for the current field, or the pre-edit interrupt flag is set for the field the cursor is moving to.

FUNCTION KEY TRANSLATION

ESP includes function key translation tables (extension .DMX), which interpret the function and special keys on your terminal and generate control characters or escape sequences. The table in the previous section uses the definitions in this file.

In addition to the keys listed above, the ESP translation tables include definitions for the following keys:

KEY NAME	CHARACTER TRANSLATED
F1	ESCAPE CTRL / A
F2	ESCAPE CTRL / B
F3	ESCAPE CTRL / C
F4	ESCAPE CTRL / D
F5	ESCAPE CTRL / E
F6	ESCAPE CTRL / F
F7	ESCAPE CTRL / G
F8	ESCAPE CTRL / H
F9	ESCAPE CTRL / I
F10	ESCAPE CTRL / J
F11	ESCAPE CTRL / K
F12	ESCAPE CTRL / L
F13	ESCAPE ESCAPE
F14	ESCAPE +
F15	ESCAPE <
F16	ESCAPE P
SHIFT / F14	ESCAPE -
SHIFT / F15	ESCAPE >
EXECUTE	ESCAPE X
NEXT SCREEN	CTRL / T
PREV SCREEN	CTRL / R

If you do not use your own function key translation files, these are the codes returned to the application when a user presses the keys listed.

The translations listed for the F1 through F12 function keys are used whether the key is shifted or unshifted.

When PCs are used as terminals via AlphaMATE 3.0 or later, AM/PC 1.2 or later, or AlphaLAN, F1 through F10 are the same as listed here. F11 through F16 on Alpha Micro terminals are equivalent to CTRL / F1 through CTRL / F6; for shifted F11 through F16, use ALT / F1 through ALT / F6.

Hints for Using the ESP Translation Table

If you choose to use the ESP translation tables in your application, rather than defining one of your own, there are some built in shortcuts which will make it easier for you to interpret the codes sent back to your application. The F1 through F12 function keys and commonly used other keys have been predefined, so you don't have to remember what codes are returned by which keys.

For assembly language programs, the TOOLBX.UNV symbol file defines the codes sent by the translation table as follows:

- F1 - F12 are $F1 through $F12 if you are using seven-bit characters, $XF1 through $XF12 for eight-bit characters.
- F13 is $MENU for seven-bit characters, $XMENU for eight-bit.
- EXECUTE is $EXEC for seven-bit, $XEXEC for eight-bit.

For AlphaBASIC programs, the following definitions are in the SYSSTD.BSI file:

- F1 - F12 are ESP'F1 through ESP'F12 if you are using seven-bit characters, ESP'XF1 through ESP'XF12 for eight-bit characters.
- F13 is ESP'MENU for seven-bit characters, ESP'XMENU for eight-bit.
- EXECUTE is ESP'EXEC for seven-bit, ESP'XEXEC for eight-bit.

For more information on interpreting the codes sent when function keys are pressed, see the section on GTSCR and TBX$GETSCR.

THE CHRCHK MACRO

The CHRCHK assembly language macro in TOOLBX.UNV makes checking for multi-character sequences, such as function keys, easier in assembly language programs. It checks for both seven-bit and eight-bit characters, allowing your application to run in either mode. The source file definition of CHRCHK is:

```
;
; Define the character checking macro. This is used mainly in routines
; that look for the characters defined above. If you use the optional REG
; REG argument, it should index the USRIMP area for the current user.

DEFINE       CHRCHK       CHAR, REG     ;
       SAVE  A6, D7                     ; Save a register.
IF     B,REG
       TOOLBX       TBX$FNDUSR
IFF
       MOV   REG, A6                    ; Get USRIMP address.
ENDC
       MOVW  UI.FLG(A6), D7             ; Get the USRIMP flags.
       BTST  #UI%NEW, D7                ; Are we checking a "new"
                                        ; character?
       BNE   10$$                       ; Yes.
       CMPB  D1, #$'CHAR                ;  No. Check 7-bit character.
       BR    20$$                       ;  Return.
10$$:        MOV   D1, D7               ; Get the character.
       AND   #^H01FF, D7                ; Save only the character.
       CMPW  D7, #$X'CHAR               ; Check 8-bit character.
20$$:        REST  A6, D7               ; Restore register.
ENDM
```

You can use CHRCHK instead of the methods discussed in Chapter 3 to check the character returned to your program from TBX$GETSCR.

USING YOUR OWN FUNCTION KEY TRANSLATIONS

The .DMX translation table file discussed above is loaded by ESP when it initializes a terminal session (the TBX$TRMINI routine or INITRM XCALL). However, before loading this file, ESP checks to see if the application already has a translation table file loaded; if it does, the .DMX file is ignored. Therefore, a translation table for a specific application always overrides the general ESP translation table.

Remember that, unless your translation table sends the same characters as the ESP tables, you cannot use the symbol names described in the previous section, but must check for the specific value sent by each key.

To load a function key translation table other than a .DMX table do the following:

Register usage and basic technique are up to the individual programmer.

```
   TOOLBX TBX$FNDUSR                  ; Index the USRIMP area.
   MOV    A6, A3                      ; Get the address.
;
; Make sure the "loaded" bit is initially clear.
;
   BCLR   #UI%XAL,UI.IFL(A3)          ; Flag XLAT as NOT loaded.
   GETIMP D.DDB, A1                   ; Get a DDB.
;
; Set up the DDB with the proper filename for the table here.
;
; Now load the table into the USRIMP area.
;
   LEA    A2, UI.XLT(A3)              ; Index where the table should go.
   MOV    A2, D.BUF(A1)               ; Set buffer index.
   FETCH  @A1, A2, F.ABS              ; Load it in.
   BNE    ERROR                       ; Not loaded.
   BSET   #UI%XAL,UI.IFL(A3); Flag XLAT as loaded.
```

When you load a translation table, be sure to set the appropriate flag, as shown above, so TBX$INITRM knows there is a table loaded and does not load the .DMX file.

When you leave the application, be sure to clear the flag, so the next ESP application knows there is no translation table loaded.

CHAPTER SIX
TRANSLATING ESP'S MESSAGES

ESP's error and status messages are in the file TBXMSG.USA. This is an assembled version of the source file TBXMSG.M68. To translate these messages, or change the messages for American English, you need to modify the TBXMSG.M68 file, then reassemble it.

TBXMSG.M68 is an assembly language program source file. Before starting this procedure, be sure to have a backup copy of the file. Be very careful while modifying it, and do not change any part of the file except those described here. If you do, you could cause serious problems or disable ESP altogether.

CHANGING THE MESSAGES

To modify TBXMSG.M68, log into ESP: and call it up using AlphaVUE or AlphaXED.

Comments in the TBXMSG.M68 file indicate which section of the file contains the messages you can change. The messages are near the middle of the file, on lines containing the macro MSGTXT. The actual text is contained in angle brackets following MSGTXT. For example:

```
TE.NUI:  MSGTXT  <User impure area unavailable>
```

You can modify the message text any way you want, making it longer or shorter if necessary, but be sure it is enclosed with angle brackets, and ***do not change any other part of the line***.

Some messages have a second set of angle brackets, containing codes which begin DE$. Do not change these codes! They control when and how the messages display.

TRANSLATING THE MESSAGE FILE

If you want to create a version of the message files for a language other than American English, you must make one other change to TBXMSG.M68.

Near the beginning of the file is a line like this:

```
OBJNAM 0, 0, [USA]
```

This determines the extension of the assembled message file. Change the USA to the extension for the language you want.

ASSEMBLING THE MESSAGE FILE

Once you've made all your changes to TBXMSG.M68, you must assemble it. You can do this in either of two ways:

- Use the AlphaVUE or AlphaXED GO command.
- From AMOS command level, enter the command:

```
M68 TBXMSG RETURN
```

Either of these methods assembles the message file, giving the resulting object file the extension specified by OBJNAM.

APPENDIX A
TOOLBOX MESSAGE CODES

This appendix lists the message codes defined in the toolbox and their corresponding messages.

MESSAGE CODE	MESSAGE TEXT
TE$IFM	Insufficient free memory available.
TE$CSO	Please contact your system operator.
TE$NUI	Cannot locate User Impure area.
TE$IVP	The password you entered does not match.
TE$NOP	Option unavailable - contact system operator.
TE$IVS	The item you selected is not recognized - please try again.
TE$INE	Warning: The item you selected is not recognized.
TE$EOS	Search concluded.
TE$EWL	Error while loading...
TE$EWR	Error while reading...
TE$UIC	Unrecognized .INI file command.
TE$WFR	Please wait ... report in process.
TE$NIP	No items printed.
TE$NWD	Terminal does not support changing of display width.
TE$IUI	Improper user ID.
TE$ISC	Your security clearance is not sufficient for the selected option.
TE$NHA	I’m sorry, there is no further help available.
TE$MIS	The maximum number of items have already been selected.
TE$IPM	Insufficient Tool Box memory pool resources - Contact System Operator.
TE$IDE	Insufficient data entered.
TE$IDT	Invalid date format.
TE$INM	Invalid numeric entry.
TE$XIA	Invalid expression argument.
TE$XSE	Expression syntax is invalid.
TE$XUP	Expression parentheses are unmatched.
TE$NPD	No printers defined for this operation.

MESSAGE CODE	MESSAGE TEXT
TE$DV0	Divide by zero.
TE$VNA	Value not available.
TE$IVL	Illegal value.
TE$PNF	Printer not found.
TE$PFN	File not found in printer queue.
TE$PRD	Printer request denied - wrong CPU.

APPENDIX B
THE USER IMPURE AREA

As described in the *ESP Installation Instructions* and the *Overview of ESP*, when TOOLBX.SYS is loaded into system memory, it reserves approximately 11K of memory per ESP user to store information specific to that user. This is called the user impure area, or USRIMP. This appendix describes the internal structure of USRIMP.

The information kept in the user impure area can be broadly divided into three types:

- Information used internally by ESP, which an applications programmer does not need to read and should not change.
- Information the applications programmer may need to read, but doesn't need to write.
- Information you may need to both read and write from your application.

In general, any reading or writing in the user impure area should be done using the appropriate toolbox assembly language subroutine or AlphaBASIC XCALL. The rare instances where you may want to access USRIMP information directly are noted in this appendix.

If reading or writing USRIMP directly, always use the offset labels! Since some areas may change size depending on the toolbox version, using a direct numeric offset could corrupt the toolbox or other areas of system memory.

The rest of this appendix describes the contents of USRIMP, in the order in which they are stored in the impure area. For any location you may want to read or write, we give the offset label, size, and contents in a columnar format. The size of some offsets is set to the value of a constant set elsewhere in the toolbox. If you need to refer to this size, use the constant, not a numeric literal size.

For areas which are used internally by ESP only, we give only general information.

BLOCK LINKS

The first four bytes point to the next user in the chain of user impure areas.

The next four bytes point to the JCB for this user.

DO NOT ALTER this information.

USER OPTIONS

You can read or write the user options using the GETUSR or SETUSR XCALL from AlphaBASIC or the TBX$GTUSER or TBX$STUSER assembly subroutine. Much of this information can also be set with SIGNIN. The specific options are:

UI.FLG	2	User flags. The individual flags are: UI$MDM User is on a modem. UI$NSP Do NOT spool ESP screens. UI$NEW Application uses eight-bit characters. UI$CCA Control-C is active during SIGNIN. You can refer to the first three flags as bit number flags using the names UI%MDM, UI%NSP, and UI%NEW.
UI.RDS	1	Read security level.
UI.WTS	1	Write security level.
UI.TMO	2	Timeout period in seconds.
UI.TOC	1	Timeout character for seven-bit applications. For eight-bit applications, use UI.TO8, described below.
UI.DBG	1	Reserved for debug flags. None are currently defined.
UI.UNM	S..UNM	User name.
UI.BCL	1	Default background color.
UI.FCl	1	Default foreground color.
UI.CNM	S..CNM	Company name.
UI.HLP	S..FNB	Default help file file specification.
UI.DFP	4	Default printer name.

WORK AREAS

These three labels mark work space used by some of the toolbox routines. Your own routines can use these areas, but you should make sure they are not already being used before accessing them.

UI.DDB	S..DDB	Miscellaneous access DDB.
UI.RBF	S..RBF	Miscellaneous access record buffer for DDB.
UI.WRK	S..UIW	Miscellaneous work area.

ACCUMULATORS

The next section of the impure area contains the values for this user's ESP accumulators. These accumulators are described in the *SEDIT User's Manual*. You can also read and write the accumulators using TBX$STUSER, TBX$GTUSER, TBX$GETACM, TBX$SETACM, and the SETUSR and GETUSR XCALLs.

There are currently 512 accumulators. You should access them only using the routines listed above.

EXPRESSION STACK AREA

This is a work area used by the EVLFLD subroutine.

UI.STB	S..STK	The base of the expression stack.
UI.STK	0	The top of the stack.

PRINTER DEFINITIONS

The first part of this section contains printer definitions loaded from the SYS:PRTDEF.lan file by TBX$TRMINI or the INITRM XCALL. These definitions can also be loaded by the TBX$PRTDEF routine, which can load printer definitions from any file. The TBX$FNDPRT routine can find a particular printer in this table, or an available slot to which you can write another printer definition.

The label for the beginning of the printer definitions is UI.PDF. The size of the printer definition area is S..PDF*MX$PRT. S..PDF is the size of a single printer definition; MX$PRT is the maximum number of printers.

Each printer definition has the following format:

PD.NAM	4	The printer name, packed RAD50.
PD.DNM	4	The printer driver name, packed RAD50.
PD.DVR	4	The printer driver address.
PD.DSC	S..PDS	The description of the printer.
PD.WHL	1	The print wheel number.
PD.FLG	1	Printer flags: PD$SFD Printer has a sheet feeder. PD%OSP Printer uses memory based spooler.
PD.CPU	4	CPU-ID for remote printer.
PD.SPR	8	Spare area.

Following the printer definitions is the spooler definition block, which contains the currently selected printer and spooler options. The offset label for it is UI.SPL, and its size is S..SPL. Its structure is:

SP.PNM	4	Name of the selected printer, packed RAD50.
SP.CPU	4	The CPU-ID of the spooler.
SP.PSW	4	Positive switches (see below).
SP.NSW	4	Negative switches (see below).
SP.CPY	2	Number of copies requested.
SP.BAN	4	Banner pointer.
SP.LPP	2	Line per page setting.
SP.WID	2	Width setting.
SP.FRM	4	Form name.
SP.PRI	2	Priority.
SP.RDT	4	Run date.
SP.RTM	4	Run time.
SP.RST	2	Restart option.
SP.SPG	2	Start page.
SP.EPG	2	End page.
SP.LMT	2	Limit.

When setting spooler switches, you can force an option to be on by setting the switch in SP.PSW, or force it to be off by setting it in SP.NSW. If you don't set it in either area, the default is used, which may be either on or off. The switches are:

SP$CPY	Number of copies
SP$DEL	Delete after printing.
SP$BAN	Print banner.
SP$HDR	Print header.
SP$FF	Form feed.
SP$LPP	Set lines per page.
SP$FRM	Set form type.
SP$INF	Inform user when printed.

The last thing in the printer area is the current page number:

UI.PAG	2	Current page number for reports.

TRANSLATION TABLE

This is the area where a function key translation table is loaded. TBX$TRMINI or XCALL INITRM loads the ESP translation table in this location unless your application has already loaded its own table.

The label for this location is UI.XLT. The size of the translation table area is S..XLT.

If you load your own translation table, be sure to set the UI%XAL flag (defined below) to tell TBX$TRMINI (or INITRM) there is a table loaded. Otherwise, the ESP translation table will be loaded over yours. When your application finishes, you need to clear this flag.

CURRENT SCREEN INFORMATION

This area contains information about the current screen and cursor position. This information is updated by the screen handling routines and various subroutines use it as necessary. You should never write directly to this area.

UI.CSC	4	Pointer to the address of the current screen.
UI.CFL	2	Number of the field currently being edited.
UI.SCL	4	Pointer to the list of active ESP screens.
UI.PUP	PU.SIZ	Information about current pop-up menu field.
UI.SVF	S..SVF	Saved field area.
UI.CRO	1	Row offset of current cursor position.
UI.CCO	1	Column offset of current cursor position.

BUSY INDICATOR VARIABLES

The next eighteen bytes contain various options and settings controlling the "busy" indicator. This indicator can be either a percentage counter or a revolving line. Various toolbox subroutines, such as TBX$BSYTGL and TBX$BSYOFF, read and write this information. You should never need to access it without using these routines.

SIGNIN INFORMATION

The next four bytes contain information relating to the SIGNIN security system. This information is loaded by SIGNIN. You can also read and write these offsets using TBX$GTUSER and TBX$STUSER (GETUSR and SETUSR from AlphaBASIC).

UI.SGF	1	SIGNIN flags. Only one is currently defined: UI%SGA Is SIGNIN active?
UI.SP1	1	Reserved.
UI.CMP	1	SIGNIN company number.
UI.MNU	1	SIGNIN menu selection number.

INDIRECT INPUT INFORMATION

You can read and write the indirect input information in this area using TBX$GTUSER and TBX$STUSER.

UI.IIF	1	Indirect input flags. The only one defined is: II%KIN User in is keyboard input mode.
UI.BKC	1	Break character to use for ^_K during input. For eight-bit applciations, use UI.BC8, described below.
UI.IIC	4	Pointer to the first block in the indirect input chain.

EXTERNAL LINKAGES

This section contains pointers to various areas outside USRIMP. There is only one you should read or write from your application:

UI.UMT	4	Pointer to the current ESP application's externally loaded user message table.

MISCELLANEOUS FLAGS

UI.IFL 1 Internal flags. Except for UI%XAL, these are set as appropriate by ESP, and your application should not set them. They are used by TBX$TRMINI and other subroutines.

- UI%TMO Timeout in progress.
- UI%XAL Function key translation table already loaded. As noted above, you should set this flag if you load your own translation table before using TBX$TRMINI.
- UI%PRT Printer definitions already loaded.
- UI%GRF Graphics device definitions already loaded.
- UI%CMD An input command prefix is in progress.

(ROB: What does that mean?)

UI.SP2 1 Reserved.

UI.SSD 2 Software security device (SSD) number. You can read this number with the GETUSR XCALL.

UI.KSC 2 Keystroke count. This is incremented by GETCHR. It is not used by ESP; it is kept only for applications which want to know this information.

KEYSTROKE RECORD INFORMATION

The next ten bytes hold information relating to keystroke recording (macros). Some of this room is reserved for future use. You should not write to this area, and should not need to access it directly.

GRAPHICS DEVICE DEFINITIONS

This section contains graphics device definitions loaded by the TBX$GRFDEF routine from SYS:GRFDEF.lan or any application-specific file. The TBX$FNDGRF routine can find a particular device in this table, or an available slot to which you can write another graphics device definition.

The label for the beginning of the definitions is UI.GDF. The size of the graphics device definition area is S..PDF*MX$GRF. S..PDF is the size of a single definition; MX$GRF is the maximum number of graphics devices.

The definition for each graphic device is the same as that described above for printers. The label names begin with GD instead of PD.

REGISTER STORAGE

The user impure area includes an area where you can temporarily store the A0 - A5 registers if necessary. This is included mainly for applications which use AMSORT.

UI.REG	24.	Temporary storage space for A0 - A5.

EIGHT-BIT CHARACTER STORAGE

The last information in USRIMP are alternate definitions for the timeout character and indirect input break character. The offset labels described above are used for seven-bit applications; these labels should be used for eight-bit applications.

UI.TO8	2	Timeout character for eight-bit applications. For seven-bit, use UI.TOC.
UI.BC8	2	Indirect input break character for eight-bit applications. For seven-bit, use UI.BKC.

These definitions are followed by six bytes of reserved space.

DOCUMENT HISTORY

REVISION	DATE	DESCRIPTION
01	8/31/95	Inocorporate all outstanding revisions. Release for AlphaCD only.
00	6/4/92	New document for ESP 2.1 inclusion in AMOS 2.2A. Replaces *TOOLBOX Programmer's Manual*. General update; reformat; add BASIC XCALLs, general guidelines, function key handling.

INDEX

A

Account
  changing 4-35
  reading current 4-9
Accumulators
  clearing 3-26
  reading 4-9
  setting 4-36
ALERT 4-1
Allocating a memory packet 3-14
  in shared memory pool 3-16

B

Background color
  reading user default 4-8
  setting 4-34
  setting for field 4-30
Binary value
  getting from screen field 3-7
  setting into field 3-19
Blinking a field 4-30

C

Changed fields
  redisplaying 3-5
Changing field position 4-31
Changing memory packet size 3-14
Character
  accepting next 3-7
  getting from keyboard 4-13
CHKTBX 3-1
Clearing
  area of the screen 4-2
  fields 4-3
Clearing a block of fields 3-4
Closing a screen 3-4, 4-3
CLRBLK 4-2
CLRFLS 4-3
CLSSCR 4-3
Colors
  reading user defaults 4-8
  setting 4-34
  setting for field 4-30
Company name
  reading 4-8
  setting 4-34
Company number
  reading 4-8
  setting 4-34
Control characters 5-1
Conventions used in this book 1-2
Counting the fields on a screen 4-38

D

Date value
  getting from field 3-8
  Julian 4-21
Deallocating a memory packet 3-15
  in shared memory pool 3-17
Debug level
  reading 4-9
  setting 4-36
Default printer name
  getting 4-7
  setting 4-34
Device
  changing 4-35
  reading current 4-9
Displaying
  error messages 4-1
Displaying a message 3-3, 3-5, 4-3

E

Eight-bit character mode
  selecting 2-1
Eight-bit characters
  and function keys 5-3
  selecting 3-26, 4-18, 4-33
ERRDSP 4-3
Error message
  displaying 4-1
Error messages displayed by ESP A-1

F

FETCH 4-4
  when to use 2-2
Field
  allowing 4-37
  changing screen position 4-31
  clearing 4-3
  clearing a block of 3-4
  counting number on a screen 4-38
  getting binary value from 3-7
  getting date value from 3-8
  getting floating point value from 3-8
  getting pop-up item number 3-9
  getting security status 4-5
  getting string value from 3-11
  getting value from 4-9
  getting value from yes/no 4-10
  getting yes/no value from 3-11
  hiding 3-13, 4-17
  redisplaying after change 3-5
  setting all for redisplay 4-31
  setting binary value 3-19
  setting display characteristics 4-30

setting floating point value 3-20
setting pop-up item for 4-32
setting read/write security 4-32
setting string value 3-21
setting to blinking 4-30
setting to reverse video 4-30
setting value 4-36
setting yes/no value 3-21
setting yes/no value for 4-37
showing 3-22, 4-17
skipping 3-22, 4-37
underlining 4-30
File
opening temporary 4-25
sizing sequential 3-23, 4-38
Finding free area in shared memory pool 3-17
Flag types 3-2
Floating point value
getting from field 3-8
setting into field 3-20
Foreground color
reading user default 4-8
setting 4-34
setting for field 4-30
Free memory
finding in shared memory pool 3-17
measuring 3-15
Function keys 5-2
and eight-bit characters 5-3
and the ESP translation table 5-3
using your own translation table 5-5

G

GETFSC 4-5
GETPUI 4-6
GETUSR 4-6
element numbers 4-7
GETVAL 4-9
GETYNF 4-10
Graphic conventions 1-2
GTCHR 4-13
GTSCR 4-13
and interrupts 4-15, 4-16
and screen output 4-16
and special keys 4-15, 5-1
information returned by 2-2
passing an editing character 2-2
returning to application 4-15
when to use 2-2
Guidelines for programming 2-1

H

Help file
reading default 4-8
Help file default
setting 4-34
HIDFLD 4-17
Hiding a field 3-13, 4-17

I

Indirect input
clearing level 4-35
reading flags 4-8
reading level 4-9
selecting file 4-35
selecting string 4-35
setting iterations 4-35
Initializing memory pool 3-16
Initializing terminal session 3-26, 4-17
INITRM 4-17
when to use 2-1, 2-3
Interrupts
and GTSCR 4-15, 4-16
and TBX$GETSCR 3-10
IREAD 4-19
ISAM
reading a data record 4-19
reading the rock 4-20
ISMROK 4-20

J

JCB
setting 4-35
JLOCK
setting or clearing 4-35
Job name
reading 4-9
Job number
reading 4-9
JULCVT 4-21
Julian date 4-21
Justifying a text string 4-21
JUSTXT 4-21

K

Keystroke record file
setting 4-35

L

List box 4-22
Loading a screen file 4-4
Loading screen file from disk 3-6
LOKSER flag
reading 4-9
LSTBOX 4-22

M

Memory
allocating using assembly language 2-1
Memory packet
allocating 3-14
allocating in shared memory pool 3-16
changing size 3-14
deallocating 3-15
deallocating in shared memory pool 3-17
Memory pool
initializing 3-16

measuring free memory 3-15
shared 3-16, 3-17
Message
displaying 3-3, 3-5, 4-3
Messages displayed by ESP A-1
Modifying ESP's messages 6-1
MSGLOG.FIL 4-35

O

On-line help 4-38
Opening a screen 3-18, 4-25
Opening a temporary file 4-25
OPNSCR 4-25
when to use 2-2
OPNTMP 4-25

P

Page number
reading 4-9
setting 4-35
Pop-up item
getting number from field 3-9
selecting value 3-20
setting for field 4-32
Pop-up item field
reading 4-6
Printer
reading default name 4-7
selecting 4-27
setting default name 4-34
Printing a screen 3-18, 4-25
Processing an ESP screen 3-9, 4-13
PRTSCR 4-25
PRTSEL 4-27

R

Random file
reading records 4-28
Read security
setting for field 4-32
Reading an ISAM data record 4-19
Reading random file records 4-28
Reading the ISAM rock 4-20
Registers
preserving and destroying 3-2
Reverse video
setting for field 4-30
Root account
reading 4-9
Root device
reading 4-9
RREAD 4-28

S

Screen
changing a field's position 4-31
clearing block 4-2
closing 3-4, 4-3
counting the number of fields 4-38
opening 3-18, 4-25
printing 3-18, 4-25
processing 3-9, 4-13
setting all fields to redisplay 4-31
updating display 3-5
Screen file
loading 4-4
loading from disk 3-6
Screen output
and GTSCR 4-16
Security
setting for field 4-32
Security status
getting for field 4-5
Selecting a printer 4-27
Selecting pop-up item value 3-20
Sequential file
sizing 4-38
Sequential file sizing 3-23
SETDFC 4-30
SETDFP 4-31
SETDSP 4-31
SETFSC 4-32
SETPUI 4-32
Setting all fields to redisplay 4-31
Setting user parameters 4-32
Setting value into a field 4-36
SETUSR 4-32
SETVAL 4-36
SETYNF 4-37
SIGNIN
reading ersatz specification 4-9
SIGNIN flag
reading 4-9
setting 4-36
SIGNIN menu selection
reading 4-8
setting 4-34
Sizing a sequential file 3-23, 4-38
Skipping a field 3-22, 4-37
reversing 3-3
SKPFLD 4-37
Special keys
and GTSCR, TBX$GETSCR 5-1
SSD number
reading 4-35
Status lines 3-26, 4-18
String
justifying 4-21
String value
getting from field 3-11
setting into field 3-21

T

TBX$ALERT 3-3
TBX$ALWFLD 3-3
TBX$CLRFLS 3-4
TBX$CLSSCR 3-4
TBX$DSPERR 3-5
TBX$DSPSCR 3-5
TBX$FETSCR 3-6
when to use 2-2
TBX$GETBNM 3-7
TBX$GETCHR 3-7
TBX$GETDAT 3-8
TBX$GETFV6 3-8

TBX$GETPUI 3-9
TBX$GETSCR 3-9
and special keys 5-1
checking for interrupts 3-10
TBX$GETSTR 3-11
TBX$GETYNF 3-11
TBX$GTUSER 3-12
symbol list 3-12
TBX$HIDFLD 3-13
reversing 3-22
TBX$MEMALC 3-14
TBX$MEMCHG 3-14
TBX$MEMDEA 3-15
TBX$MEMFRE 3-15
TBX$MEMINI 3-16
TBX$MPLALC 3-16
TBX$MPLDEA 3-17
TBX$MPLFRE 3-17
TBX$OPNSCR 3-18
TBX$SCRPRT 3-18
TBX$SETBNM 3-19
TBX$SETFV6 3-20
TBX$SETPUI 3-20
TBX$SETSTR 3-21
TBX$SETYNF 3-21
TBX$SHWFLD 3-22
TBX$SKPFLD 3-22
reversing 3-3
TBX$SQFLSZ 3-23
TBX$STUSER 3-24
symbol list 3-24
TBX$TRMCAP 3-25
TBX$TRMINI 3-26
options 3-26
TBXMSG.M68 file 6-1
Temporary file, opening 4-25
Terminal
initializing ESP session 3-26
Terminal capabilities, reading 3-25
Terminal driver name
reading 4-9
Terminal session
initializing 4-17
Text string
justifying 4-21
Timeout character
reading 4-8
setting 4-34
Timeout period
reading 4-7
setting 4-34
Toolbox
checking version 3-1
TOOLBX XCALL 4-38
TOOLBX.UNV 3-1

U

User expertise
reading 4-9
setting 4-35
User level
reading 4-9
setting 4-35
User name
reading 4-8
setting 4-34
User parameters
element numbers 4-7
reading 3-12, 4-6
setting 3-24, 4-32
symbol list 3-12, 3-24
User read security
reading 4-7
setting 4-33
User write security
reading 4-7
setting 4-33

V

Value
getting from field 4-9
getting from yes/no field 4-10

W

Write security
setting for field 4-32

Y

Yes/no field
getting value from 4-10
setting value for 4-37
Yes/no value
getting from field 3-11
setting into field 3-21